

Cursos Especialització: **Desplegament de productes de programari en contenidors**
Família Professional: **Informàtica i comunicacions**
Grau: **Superior**



5165: Metodologia Devops: preparació d'entorns de desplegament

Durada: **120 hores**

Xavi Blanes 2026

Índex

[Tema 1: Metodologies de desenvolupament i CI/CD en DevOps](#)

[Tema 2: Infraestructura TI: On-Premise vs. Cloud](#)

[Tema 3: Models d'implementació i desplegament en el núvol](#)

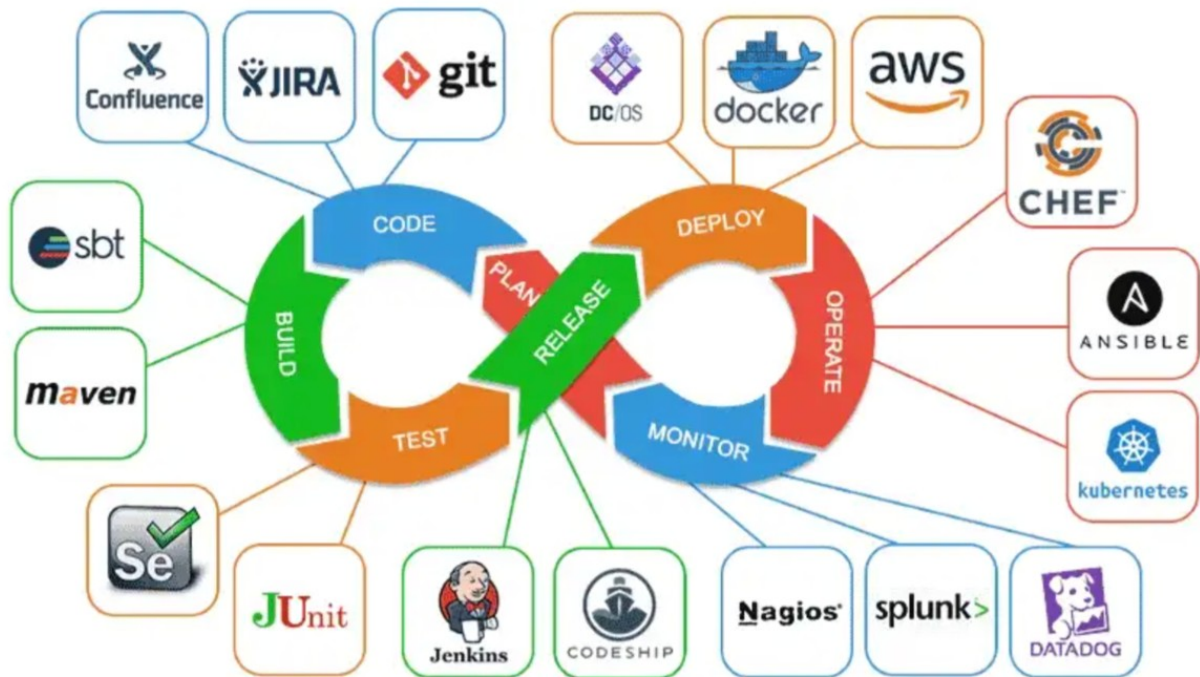
[Tema 4: Models de servei en l'arquitectura de núvol \(XaaS\)](#)

[Tema 5: Gestió de repositoris de codi, qualitat i seguretat en DevOps](#)

[Tema 6: Infraestructura com a Codi \(IaC\)](#)

[Tema 7: Gestió i monitorització en DevOps](#)

Tema 1: Metodologies de desenvolupament i CI/CD en DevOps



1. Metodologia tradicional en cascada (*Waterfall*)
2. Metodologies àgils de desenvolupament (*Agile*)
3. Ecosistema d'eines de gestió de projectes
4. La cultura DevOps en el marc àgil
5. El cicle de vida del programari en contenidors

1. Metodologia tradicional en cascada (*Waterfall*):

La metodologia tradicional en cascada (*Waterfall*) és un enfocament de gestió de projectes lineal i seqüencial, on cada fase del desenvolupament del programari s'ha de completar abans de començar la següent. Creat originalment per Winston Royce l'any 1970, aquest model assumeix que els requisits del projecte són estables i completament coneguts des del principi.

Organització del model en cascada

L'organització d'aquest model es basa en una estructura rígida i documentada. No es preveu el retorn a fases anteriors: un cop es tanca una etapa, es lliura la documentació o el programari resultant a l'equip de la següent fase. Els equips solen estar altament especialitzats per departaments (analistes, dissenyadors, programadors, testers).

Fases seqüencials del cicle de vida

El model es divideix en cinc fases principals que s'executen en un ordre estricte de dalt a baix:

- **1. Requisits (Anàlisi):** Es recullen i documenten totes les necessitats del client i els objectius del programari. El resultat és un document d'especificació de requisits (SRS) detallat i signat pel client.
- **2. Disseny:** Els arquitectes de programari tradueixen els requisits en una estructura tècnica. Es defineix l'arquitectura del sistema, les bases de dades, els diagrames de flux i les interfícies d'usuari.
- **3. Implementació (Codificació):** Els programadors escriuen el codi real del programari basant-se en els documents de disseny de la fase anterior. Es creen els primers mòduls i programes individuals.
- **4. Verificació (Proves):** Es posa en marxa tot el sistema integrat per buscar errors, fallades de rendiment o desviacions respecte als requisits inicials. L'equip de control de qualitat (QA) executa els plans de proves.
- **5. Manteniment:** Un cop el programari s'ha lliurat i està en producció, es realitzen tasques de correcció d'errors que no es van detectar abans, actualitzacions de seguretat o petites adaptacions a nous entorns.

Limitacions en el programari modern

Tot i que va ser el llistó de referència durant dècades, el model en cascada presenta greus limitacions en la indústria del programari modern, caracteritzada pel canvi constant:

- **Inflexibilitat davant del canvi:** En el programari actual, els requisits canvien constantment a causa del mercat o la competència. Cascades no permet modificar requisits a mig projecte sense un cost i un retard massius.
- **Lliurament tardà del valor:** El client no veu cap programari funcional fins a les fases finals del projecte. Si hi ha un malentès inicial, es detecta massa tard.

- **Alt risc i incertesa:** Les proves es concentren exclusivament al final. Si es descobreix un error de disseny o de concepte durant la verificació, solucionar-ho pot requerir reescriure gran part del codi.
- **Falsa sensació de progrés:** El progrés es mesura en base a documents aprovats (fites de disseny, fites de requisits) i no en programari real que funciona.

2. Metodologies àgils de desenvolupament (*Agile*):

Les metodologies àgils (*Agile*) són un enfocament iteratiu i incremental per a la gestió de projectes i el desenvolupament de programari, dissenyat per oferir valor al client de manera ràpida i contínua. A diferència del model tradicional, l'Agilisme assumeix que el canvi és inevitable i el gestiona com un avantatge competitiu en lloc d'un obstacle.

Filosofia i el Manifest Àgil

La filosofia àgil va néixer oficialment el 2001 amb la redacció del Manifest Àgil per part de 17 experts en programari. Aquest manifest es fonamenta en 4 valors core que prioritzen la flexibilitat i el factor humà sobre els processos rígids:

- **Individus i interaccions** sobre processos i eines.
- **Programari que funciona** sobre documentació exhaustiva.
- **Col·laboració amb el client** sobre negociació contractual.
- **Respondre davant el canvi** sobre seguir un pla.

D'aquests valors es deriven 12 principis, entre els quals destaquen el lliurament de programari funcional de manera freqüent (setmanes o mesos), el treball conjunt diari entre negoci i desenvolupadors, i la cerca constant de la simplicitat i l'excel·lència tècnica.

Marc de treball (Frameworks) principals

L'Agilisme és un conjunt de valors. Per a aplicar-lo, s'utilitzen marcs de treball específics. Els dos més utilitzats en el programari modern són:

1. Scrum

Es basa en el treball d'equips autogestionats en cicles de temps fixos i curts anomenats **Sprints** (habitualment d'entre 1 i 4 setmanes).

- **Rols clau:** El **Product Owner** (prioritza el que s'ha de fer), el **Scrum Master** (elimina obstacles i assegura el procés) i els **Desenvolupadors**.
- **Esdeveniments:** *Sprint Planning* (planificar el cicle), *Daily Scrum* (reunió diària de sincronització de 15 minuts), *Sprint Review* (demostració del programari al client) i *Sprint Retrospective* (reunió de millora de l'equip).
- **Artefactes:** *Product Backlog* (llista completa de tasques pendents) i *Sprint Backlog* (tasques seleccionades per al cicle actual).

2. Kanban

Es basa en la **visualització del flux de treball** de manera contínua, sense cicles de temps fixos com Scrum. Utilitza un tauler visual amb columnes que representen l'estat de les tasques (ex: *Per fer*, *En procés*, *Fet*).

- **Limitació del WIP (Work In Progress):** Es limita estrictament el nombre de tasques que es poden fer a la vegada en cada columna per evitar colls d'ampolla.
- **Focus:** Millorar l'eficiència i reduir el temps de lliurament total des que sorgeix una idea fins que es completa (*Lead Time*).

Contraposició: Agile vs. Model Tradicional (Waterfall)

Característica	Model Tradicional (Waterfall)	Metodologies Àgils (Agile)
Enfocament	Lineal, seqüencial i predictiu.	Iteratiu, incremental i adaptatiu.
Gestió del canvi	Molt difícil i costosa un cop començat el projecte.	Benvinguda. S'adapta a cada cicle o sprint.
Lliurament de valor	Al final del projecte (procés de tot o res).	Continu i de forma primerenca des de les primeres setmanes.
Rol del client	Intervé al principi (requisits) i al final (proves).	Integrat en l'equip, col·laboració constant.
Risc	Alt. Els errors greus es detecten a la fase de verificació.	Baix. Es detecten desviacions ràpidament a cada retrospectiva o demo.
Mètrica d'èxit	Compliment estricte del pla i el pressupost.	Entrega de programari funcional que aporta valor real.

3. Ecosistema d'eines de gestió de projectes

L'ecosistema d'eines de gestió de projectes es divideix segons el model de desenvolupament utilitzat, oferint funcionalitats específiques per optimitzar la planificació predictiva o la col·laboració adaptativa. L'elecció de l'eina depèn directament de la naturalesa del projecte, l'estructura de l'equip i la flexibilitat requerida.

Eines per a metodologies tradicionals

Les eines tradicionals estan orientades a la planificació a llarg termini, el control rigorós de calendaris, la gestió de costos i la dependència estricta entre tasques.

- **Diagrama de Gantt:** És la representació gràfica per excel·lència de les metodologies tradicionals. Mostra les tasques en línies de temps horitzontals, definint clarament quines tasques són correlatives (una no pot començar fins que no acabi l'anterior, anomenat relacions de precedència) i visualitzant el **camí crític** del projecte.
- **Microsoft Project:** És l'estàndard de la indústria per a la gestió de projectes tradicionals. Permet definir el cronograma detallat, assignar recursos humans i materials de manera complexa, calcular costos estimats vs. reals i controlar les desviacions respecte a la línia base del pla original.

Eines per a metodologies àgils

Les eines àgiles estan dissenyades per a la visibilitat immediata, la col·laboració diària i l'adaptació constant, prioritzant la gestió de llistes de tasques dinàmiques (*backlogs*) i indicadors de flux.

- **Jira (Atlassian):** És l'eina més potent i utilitzada en la indústria del programari modern per a Scrum i Kanban. Permet gestionar històries d'usuari, configurar fluxos de treball automatitzats i personalitzats, planificar *sprints* i extreure mètriques de rendiment com el gràfic de cremada (*burndown chart*) o de velocitat.
- **Trello:** Una eina altament visual i senzilla basada exclusivament en el mètode Kanban. Utilitza un sistema de llistes i targetes (fàcils d'arrossegat i deixar anar) que és ideal per a equips petits o projectes que busquen una gestió de tasques fluida, intuïtiva i sense configuracions complexes.
- **Azure Boards (Microsoft):** Integrat dins de la suite Azure DevOps, és una solució robusta per a equips que utilitzen entorns de Microsoft. Ofereix taulers Kanban, planificació de *sprints* per a Scrum i enllaça directament les tasques de gestió amb el codi font, les branques de Git i els desplegaments en el núvol.

Comparativa d'enfocament tecnològic

Atribut	Enfocament Tradicional (MS Project / Gantt)	Enfocament Àgil (Jira / Trello / Azure Boards)
Objectiu principal	Controlar el compliment d'un calendari fix.	Gestionar el valor lliurat i el flux de treball.
Element central	Les dates d'inici/fi i les dependències.	Les històries d'usuari, tasques i el tauler visual.
Mètriques	Desviació de temps, costos i ús de recursos.	Velocitat de l'equip, temps de cicle (<i>cycle time</i>).
Públic objectiu	Gestors de projectes (Project Managers).	Tot l'equip de desenvolupament i el Product Owner.

4. La cultura DevOps en el marc àgil

La cultura DevOps és una evolució natural del moviment àgil que trenca la barrera històrica entre el desenvolupament (Development) i les operacions de sistemes (Operations). Mentre que les metodologies àgils ajuden els equips a dissenyar i programar programari en cicles curts, DevOps estén aquesta agilitat fins al lliurament final al client, automatitzant tot el camí per fer-lo ràpid, segur i repetible.

El cicle de vida DevOps

El cicle DevOps es representa habitualment com un símbol d'infinit (∞), reflectint que el desenvolupament de programari és un procés de millora contínua i aprenentatge constant. Aquest cicle es divideix en vuit fases interconnectades:

- **1. Planificació (Plan):** Es defineixen els objectius del programari, les tasques i els requisits de l'usuari (fase molt lligada a Scrum o Kanban).

- **2. Codificació (Code):** Els desenvolupadors escriuen el codi i el puguen a un repositori centralitzat utilitzant eines de control de versions (com **Git**).
- **3. Construcció (Build):** El codi es compila, es preparen les dependències i es genera un executable o paquet instal·lable.
- **4. Proves (Test):** S'executen bateries de proves automatitzades per assegurar que el programari no conté errors ni trenca funcionalitats anteriors.
- **5. Alliberament (Release):** El programari es valida i es deixa a punt per a la seva distribució (fase de transició cap a producció).
- **6. Desplegament (Deploy):** S'instal·la el codi de forma automàtica als servidors o entorns de producció on el faran servir els usuaris reals.
- **7. Operació (Operate):** El programari s'executa en viu; es gestionen la infraestructura, els servidors, l'escalabilitat i la seguretat en el núvol.
- **8. Monitorització (Monitor):** Es recullen dades en temps real sobre el rendiment de l'aplicació i el comportament de l'usuari per detectar errors i planificar millores (reiniciant el cicle).

El paper fonamental de CI/CD

El cor tècnic de la cultura DevOps és la creació d'una canonada d'automatització (pipeline), dividida en dues pràctiques clau:

a) Integració Contínua (CI - Continuous Integration)

És la pràctica d'integrar els canvis de codi de tots els desenvolupadors en una branca principal de manera freqüent (diverses vegades al dia).

- **Com funciona:** Cada cop que un programador puja codi, un servidor de CI automatitzat (com **GitHub Actions**, **Jenkins** o **GitLab CI**) detecta el canvi, compila el projecte (*Build*) i executa les proves (*Test*).
- **Benefici principal:** Permet detectar errors de manera immediata (en minuts) en lloc d'esperar a fases finals del projecte, reduint dràsticament els problemes d'integració de codi.

b) Desplegament Continu (CD - Continuous Deployment / Delivery)

És l'extensió de la CI que s'encarrega d'automatitzar el trasllat del codi provat cap als entorns reals de treball. Hi ha una subtil diferència conceptual:

- **Lliurament Continu (Continuous Delivery):** El codi passa totes les proves automàtiques i queda llistat per ser desplegat a producció, però el pas final requereix una **aprovació manual** d'un responsable.
- **Desplegament Continu (Continuous Deployment):** Tot el procés està totalment automatitzat. Si el codi passa les proves de CI amb èxit, es desplega directament a producció **sense intervenció humana**.

- **Benefici principal:** Redueix el temps de llançament al mercat (*Time-to-Market*) d'hores o dies a només uns minuts, minimitzant el risc dels desplegaments massius manuals que abans es feien de matinada o els caps de setmana.

5. El cicle de vida del programari en contenidors

El cicle de vida del programari en contenidors representa l'estàndard actual de la indústria per garantir que una aplicació s'execute de la mateixa manera en qualsevol entorn, ja sigui l'ordinador d'un desenvolupador, un servidor de proves o el núvol de producció. Mitjançant els fluxos de CI/CD, aquest procés s'automatitza completament per eliminar les tasques manuals i els errors de configuració.

Concepte d'empaquetament d'aplicacions

Tradicionalment, per executar una aplicació calia instal·lar manualment el sistema operatiu, les llibreries, les dependències i el codi en cada servidor. Això provocava el típic problema de: *"A la meva màquina funciona, però a producció falla"*.

Els **contenidors** resolen això mitjançant un enfocament d'empaquetament avançat:

- **Què és un contenidor?** És una unitat de programari lleugera, autònoma i executable que inclou absolutament tot el necessari per funcionar: el codi, l'entorn d'execució (*runtime*), les eines del sistema, les llibreries i la configuració.
- **Aïllament a nivell de Sistema Operatiu:** A diferència de les màquines virtuals (que necessiten un sistema operatiu sencer per a cadascuna), els contenidors **comparteixen el nucli (kernel) del sistema operatiu amfitrió**. Això els fa extremadament lleugers, ràpids d'arrencar (en mil·lisegons) i eficients en l'ús de memòria i CPU.
- **Imatges vs. Contenidors:** Una **imatge** (com una imatge de Docker) és un fitxer estàtic, immutable i de només lectura que conté la recepta de l'aplicació. Un **contenidor** és la instància viva i en execució d'aquella imatge.

Com els fluxos de CI/CD automatitzen els contenidors

L'ús de contenidors transforma els *pipelines* de CI/CD en processos molt més predictibles i fluidos. El flux teòric d'automatització funciona de la següent manera:

[Codi + Dockerfile] → (CI: Build Imatge → Proves) → [Registre d'Imatges] → (CD: Desplegament) → [Entorn Execució]

Fase d'Integració Contínua (CI): Construcció i Validació

- **Definició de la recepta:** El desenvolupador puja al repositori de codi (Git) el codi de l'aplicació juntament amb un fitxer de configuració de la imatge (com un **Dockerfile**). Aquest fitxer descriu el pas a pas de com s'ha de muntar el contenidor.
- **Construcció automàtica (*Build*):** El servidor de CI detecta el canvi en el repositori i construeix automàticament la imatge del contenidor executant les instruccions del fitxer de configuració.

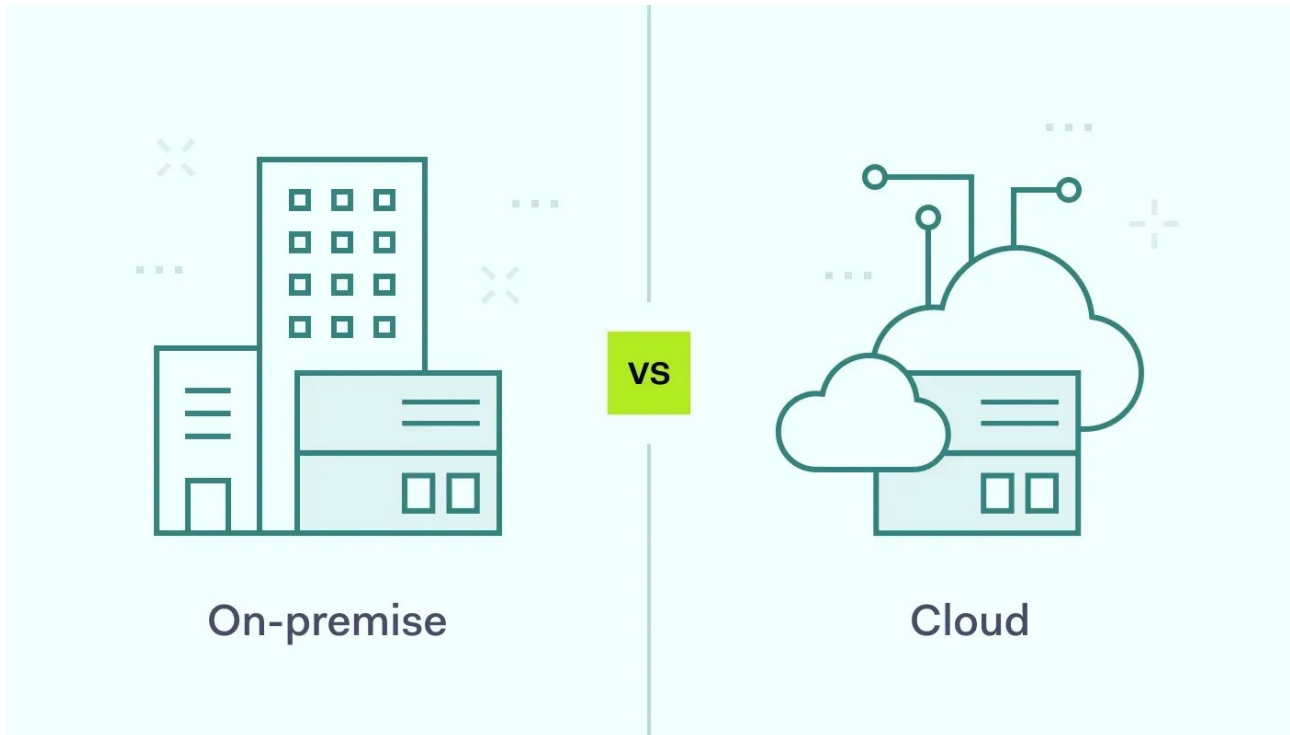
- **Execució de proves (Test):** Les proves automatitzades s'executen **dins** d'aquell mateix contenidor que s'acaba de crear. Si les proves fallen, el procés s'atura. Si tenen èxit, sabem del cert que la imatge és completament segura i estable.
- **Publicació al Registre:** La imatge generada es puja i s'emmagatzema en un repositori d'imatges centralitzat anomenat **Registry** (com Docker Hub, GitHub Packages o Azure Container Registry). Cada imatge es guarda amb una etiqueta o versió única (per exemple, la-meva-app:v1.2.0).

Fase de Desplegament Continu (CD): Distribució i Execució

- **Notificació de nova versió:** El flux de CD detecta que hi ha una nova imatge validada disponible al registre.
- **Actualització de l'entorn:** El sistema de CD (com GitHub Actions o eines específiques com ArgoCD) es connecta a l'entorn de destinació (que pot ser un servidor simple o un orquestrador de contenidors complex com **Kubernetes**).
- **Desplegament sense interrupció (Zero Downtime):** L'orquestrador descarrega la nova imatge del registre, arranca els nous contenidors i, de manera gradual, apaga els vells. Com que les imatges són immutables, l'entorn de producció utilitzarà exactament el mateix paquet de programari que va passar les proves a la fase de CI, eliminant qualsevol sorpresa d'última hora.

--ooOOoo--

Tema 2: Infraestructura TI: On-Premise vs. Cloud



1. El canvi de paradigma
2. Model de negoci de la computació en el núvol
3. Anàlisi cost-benefici del Cloud
4. Infraestructura de núvol vs. Arquitectura de núvol
5. Arquitectura física i virtual del núvol
6. Caracterització de recursos en el núvol

1. El canvi de paradigma

El pas de la infraestructura tradicional a la tecnologia cloud representa un canvi cultural i tecnològic profund. A continuació, es desenvolupa conceptualment aquest canvi de paradigma:

Definició conceptual dels models

- **Infraestructura On-Premise (Local):** És el model tradicional on l'empresa posseeix, allotja i gestiona física i lògicament tot el seu maquinari i programari. Els servidors, l'emmagatzematge i les xarxes es troben a les **instal·lacions pròpies** (un centre de dades local o una sala de servidors).
- **Infraestructura Cloud (En el núvol):** És el model basat en el consum de recursos de computació a petició a través d'Internet. La infraestructura física és propietat d'un proveïdor extern (com AWS, Microsoft Azure o Google Cloud), que en gestiona el manteniment, mentre que l'empresa en lloga l'ús de forma virtual.

El canvi de paradigma: De la propietat al servei

Aquest canvi es pot resumir en tres grans transformacions conceptuals:

- **De producte a servei (As-a-Service):** L'empresa ja no compra caixes ni components físics, sinó que contracta capacitats de computació, memòria i emmagatzematge com si fos un subministrament públic (com la llum o l'aigua).
- **De la rigidesa a l'elasticitat:** En l'entorn *On-Premise*, créixer implica comprar nou maquinari, configurar-lo i esperar setmanes. Al *Cloud*, la infraestructura és elàstica: es pot augmentar o disminuir la capacitat de forma automatitzada i en qüestió de minuts segons la demanda real.
- **De la inversió al cost operatiu (CapEx vs. OpEx):** Es passa d'un model de grans inversions inicials de capital (**CapEx**) en equips que es devaluen, a un model de despeses operatives (**OpEx**) on es paga exactament pel que es consumeix (pagament per ús).

Comparativa directa

Característica	Infraestructura On-Premise	Infraestructura Cloud
Ubicació física	Local, a l'empresa.	Remota, centres de dades del proveïdor.
Control i accés	Total i exclusiu sobre el maquinari.	Delegat (el proveïdor gestiona el físic).
Manteniment	A càrrec de l'equip de TI de l'empresa.	Automatitzat i gestionat pel proveïdor.
Escalabilitat	Lenta i limitada al maquinari disponible.	Immediata, pràcticament il·limitada.

2. Model de negoci de la computació en el núvol

El model de negoci de la computació en el núvol trenca completament amb la gestió financera de la tecnologia tradicional, convertint la infraestructura en un actiu dinàmic.

El principi del pagament per ús (Pay-as-you-go)

El pagament per ús trasllada el concepte dels subministraments públics (com l'electricitat) a la tecnologia.

- **Mesurament precís:** Les empreses contracten recursos de computació, memòria o emmagatzematge i paguen exclusivament pels segons, minuts o hores que aquests recursos estan actius.
- **Eliminació del malbaratament:** En un entorn local, un servidor encès durant el cap de setmana sense rebre visites costa el mateix que a ple rendiment. Al núvol, els recursos es poden apagar o programar per reduir la factura a zero quan no es fan servir.

Optimització de costos: CapEx vs. OpEx

La transició financera del model local al núvol es fonamenta en la reestructuració de les despeses tecnològiques:

- **CapEx (Capital Expenditure / Inversió de capital):** Propi de l'entorn **On-Premise**. Requereix un gran desemborsament inicial de diners per comprar servidors, llicències, sistemes de refrigeració i cablatge. Aquestes inversions es consideren actius fixos que es deprecien al llarg del temps (normalment entre 3 i 5 anys) i suposen un risc financer si el projecte no funciona.
- **OpEx (Operational Expenditure / Despeses operatives):** Propi de l'entorn **Cloud**. No hi ha inversió inicial; el cost s'integra en les despeses del dia a dia del negoci (com una subscripció o factura mensual). Això permet a les empreses reinvertir el capital inicial en el seu *core business* i tenir una major liquiditat.

Elasticitat de la demanda i eficiència financera

L'elasticitat és la capacitat del núvol per adaptar de forma automàtica i en temps real la infraestructura disponible a la demanda real de l'usuari.

- **El problema de l'aprovisionament local (On-Premise):** Les empreses es veuen obligades a dimensionar els seus servidors pensant en el (per exemple, el *Black Friday* en un comerç electrònic). Això provoca que la resta de l'any la infraestructura estigui infrautilitzada (sovint a menys del 20% de la seva capacitat), malbaratant diners.
- **La solució elàstica (Cloud):** El núvol utilitza mecanismes d'autoescalat (*Auto-scaling*). Si el trànsit d'una web augmenta de cop, el sistema crea nous servidors virtuals a l'instant. Quan la punta de feina passa, aquests servidors es destrueixen automàticament. L'empresa només paga la corba exacta de la demanda, optimitzant al màxim el cost.

Resum conceptual de l'impacte financer

Concepte	Model Tradicional (On-Premise)	Model de Negoci Cloud
Estructura de costos	Majoritàriament CapEx (Inversió inicial).	Totalment OpEx (Cost operatiu).
Previsibilitat	Costos fixos coneguts, però rígids.	Costos variables lligats al consum real.
Risc financer	Alt (si el negoci decreix, el maquinari segueix pagat).	Baix (si el negoci decreix, es redueixen els recursos).
Gestió de pics	Infraestructura sobredimensionada i infrautilitzada.	Infraestructura elàstica que s'ajusta a la corba de demanda.

3. Anàlisi cost-benefici del Cloud

Per avaluar la viabilitat de la migració al núvol, és fonamental realitzar una anàlisi cost-benefici que vagi més enllà del preu del maquinari. Aquest punt analitza com el **Cloud** transforma l'eficiència operativa de l'empresa a través de cinc vectors clau:

1. Manteniment i alliberament de recursos de TI

- **On-Premise:** L'equip de tecnologia intern s'ha d'ocupar de tasques físiques i rutinàries que no aporten valor directe al negoci: neteja de servidors, control de la climatització, canvis de discos durs fallits, actualitzacions de firmware i pedaços de seguretat físics.
- **Cloud:** Es delega el manteniment del maquinari i de la infraestructura base al proveïdor. L'equip de TI de l'empresa es deslliga de la gestió del "ferro" i pot enfocar el 100% del seu temps a la innovació, el desenvolupament de producte i l'estratègia de negoci.

2. Disponibilitat i Acords de Nivell de Servei (SLA)

- **On-Premise:** Assolir una alta disponibilitat (que els sistemes no caiguin) requereix inversions milionàries en dobles línies elèctriques, generadors dièsel i connexions a Internet redundants. Tot i així, una avaria local pot aturar l'empresa durant hores.
- **Cloud:** Els grans proveïdors ofereixen infraestructures hiperconnectades i garanteixen per contracte (**SLA**) disponibilitats extremadament altes (sovint del **99,9% al 99,99%**). Si un component falla, el sistema commuta automàticament a un altre sense interrupció del servei.

3. Redundància i Recuperació davant Desastres (Disaster Recovery)

- **On-Premise:** Tenir una còpia de seguretat de tota la infraestructura requereix duplicar el centre de dades en una ubicació geogràfica diferent (un segon local pagant lloguer, llum i maquinari mirall), un cost prohibitiu per a la majoria de Pimes.
- **Cloud:** Permet la **redundància geogràfica** amb uns quants clics. Les dades es repliquen automàticament entre diferents zones de disponibilitat o regions separades per centenars de quilòmetres. Davant d'una catàstrofe natural o incendi en un centre de dades, el servei continua actiu des d'una altra ubicació.

4. Escalabilitat (Vertical i Horitzontal)

- **On-Premise:** L'escalabilitat és finita i lenta. Si un servidor es queda sense memòria RAM o espai de disc, cal apagar-lo, obrir-lo físicament i instal·lar els nous components (escala vertical), o comprar un servidor nou (escala horitzontal).
- **Cloud:** L'escalabilitat és pràcticament il·limitada i automatitzada.
- **Vertical (Scale-up):** Es pot augmentar la potència d'una màquina virtual (més CPU o RAM) instantàniament des d'un panell de control.
- **Horitzontal (Scale-out):** Es poden afegir desenes de servidors en paral·lel per repartir la càrrega de treball en qüestió de minuts gràcies als **balancejadors de càrrega**.

5. Terminis d'aprovisionament (Time-to-Market)

- **On-Premise:** Quan una empresa vol llançar un nou projecte, el termini d'aprovisionament pot trigar **setmanes o mesos**. Inclou: sol·licitar pressupostos, aprovar la inversió, esperar el lliurament del proveïdor logístic, muntar-lo al rack, cablejar-lo i configurar-lo.
- **Cloud:** El termini d'aprovisionament es redueix a **minuts o segons**. Un desenvolupador pot aixecar una base de dades, un servidor web o un entorn de proves complet mitjançant una línia de comandes o una consola web. Això redueix dràsticament el temps necessari per llançar productes al mercat (*Time-to-Market*).

Resum de l'Anàlisi Cost-Benefici

Factor analitzat	Cost / Limitació (On-Premise)	Benefici / Valor (Cloud)
Cost de Personal	Alt cost en tasques de manteniment físic.	Eficiència en focalitzar el talent en el negoci.
Inversió en Continuitat	Molt alta per duplicar instal·lacions.	Inquieta i econòmica gràcies a la rèplica global.
Cicle d'Innovació	Lent, lligat als calendaris d'adquisició.	Immediat, permet experimentar i errar ràpid a baix cost.

4. Infraestructura de núvol vs. Arquitectura de núvol

Encara que sovint s'utilitzen com a sinònims en converses informals, en l'àmbit professional i d'enginyeria de programari existix una distinció crítica entre ambdós conceptes. El disseny de plataformes digitals eficients requereix entendre on acaben els components i on comença el disseny del sistema.

A continuació, es desenvolupa la diferenciació tècnica entre infraestructura i arquitectura de núvol:

a) Infraestructura de núvol (Cloud Infrastructure)

La infraestructura és el **"què"**: el conjunt d'elements físics i virtuals subjacents que fan possible la computació. Representa el teixit de recursos bruts que el proveïdor de núvol posa a la nostra disposició.

- **Assignació de recursos de computació:** Se centra en la provisió pura. És el procés de seleccionar i aixecar les peces bàsiques: triar el tipus de màquina virtual (nombre de vCPUs i gigabytes de memòria RAM), assignar el tipus d'emmagatzematge (discos d'estat sòlid SSD o emmagatzematge d'objectes) i configurar el direccionament IP i els encaminadors virtuals.
- **Component físic i de virtualització:** Inclou els centres de dades físics, els servidors fulla (*blade servers*), el cablatge de fibra òptica i la capa de l'hipervisor (el programari que divideix el maquinari real en màquines virtuals).

b) Arquitectura de núvol (Cloud Architecture)

L'arquitectura és **com es combinen i s'organitzen** eixos components per a resoldre un problema de negoci concret. És el plànol d'enginyeria que definix les relacions, els fluxos de dades i les regles de comportament entre els diferents recursos d'infraestructura.

- **Execució eficient d'aplicacions web:** L'arquitectura s'encarrega d'orquestrar els recursos per a garantir que l'aplicació siga ràpida, resiliència i tolerant a fallades.
- c) **Conceptes arquitectònics clau:**
- **Desacoblament de components:** Dividir una aplicació web en microserveis independents per a evitar que la fallada d'un element (per exemple, el mòdul de generació de factures) tire a terra tota la web.
- **Estratègies de memòria cau (Caching):** Decidir en quines capes del sistema es guarden còpies temporalment de les dades (utilitzant CDN com Cloudflare o bases de dades en memòria com Redis) per a no col·lapsar la base de dades principal.
- **Polítiques de resiliència:** Dissenyar circuits de fallada (*circuit breakers*) i rutes alternatives de trànsit si una zona de disponibilitat geogràfica cau.

d) Comparativa tècnica orientada al rendiment

La relació entre ambdues es comprén millor analitzant com col·laboren per a fer funcionar una aplicació web:

Criteri tècnic	Infraestructura de núvol (Els Components)	Arquitectura de núvol (El Disseny)
Objectiu principal	Proporcionar potència de càlcul, xarxa i emmagatzematge.	Garantir l'eficiència, escalabilitat i disponibilitat del sistema.
Enfocament de l'escalabilitat	Ofereix la capacitat física (més servidors disponibles per a ser usats).	Configura la lògica (quan, com i sota quines regles de trànsit s'activarà l'autoescalat).
Gestió de dades	Proporciona el volum de disc dur virtual (ex: un volum SSD de 500 GB).	Definix el model de base de dades (Relacional vs. NoSQL) i la seua replicació de lectura/escriptura.
Rol professional	Administrat per Enginyers de Sistemes / Cloud Administrators .	Dissenyat per Arquitectes de Núvol (Cloud Architects) / Devops .

e) Exemple pràctic en una aplicació web d'alt trànsit

- **L'error d'infraestructura sense arquitectura:** Si una empresa té una aplicació mal programada (monolítica i síncrona) i rep una allau d'usuaris, l'única solució d'infraestructura és comprar una màquina virtual gegant i caríssima (escala vertical). Tot i així, la web acabarà caient quan s'arribe al límit del processador.
- **La solució arquitectònica:** L'arquitecte dissenya el sistema col·locant un **balancejador de càrrega** al davant, seguit d'un grup de servidors web idèntics que creixen horitzontalment, i connectats a una cua de missatges asíncrona. L'arquitectura aprofita la flexibilitat de la infraestructura per a executar l'aplicació de manera intel·ligent, gastant només els diners necessaris en cada segon.

5. Arquitectura física i virtual del núvol

Per comprendre com el núvol converteix el maquinari real en serveis elàstics, cal analitzar la interacció entre la seua infraestructura física i la capa d'abstracció digital. Aquest punt detalla els components essencials d'aquesta arquitectura.

5.1. L'arquitectura física (El Maquinari Subjacent)

Els proveïdors de núvol construeixen centres de dades massius distribuïts globalment. Els components físics bàsics són:

- Servidors Fulla (*Blade Servers*): Equips d'alta densitat muntats en armaris (*racks*). Disposen de múltiples processadors multinucli d'última generació (CPUs/GPUs) i grans quantitats de memòria RAM.
- Sistemes d'Emmagatzematge en Xarxa (SAN/NAS): Cabines de discos d'estat sòlid (SSD) interconnectades per oferir emmagatzematge massiu d'alta velocitat, independentment del servidor on s'executi la tasca.
- Xarxes de teixit (*Fabric Networks*): Commutadors (*switches*) i routers de fibra òptica de gran amplada de banda que connecten els servidors amb latències properes a zero.

5.2. L'arquitectura virtual (El Programari de Control)

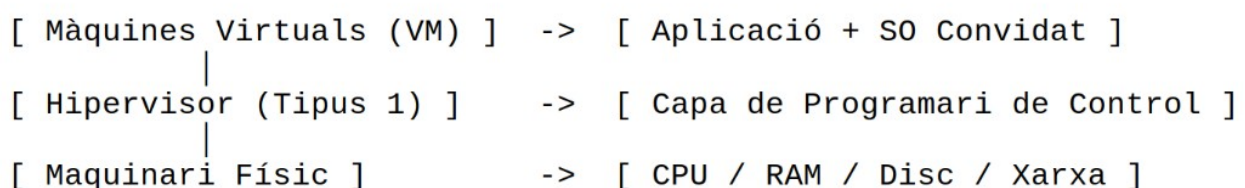
La virtualització és la tecnologia que fa possible el núvol. Permet fragmentar un servidor físic en dotzenes d'entorns virtuals aïllats anomenats Màquines Virtuals (VM).

L'Hipervisor: El cor de la virtualització

L'hipervisor (o Monitor de Màquines Virtuals) és una capa de programari especialitzat que està entre el maquinari físic i els sistemes operatius virtuals. La seva funció és interceptar les peticions de recursos i coordinar-les amb el maquinari real.

Es divideix en dos tipus conceptuals:

- Tipus 1 (Nadiu o *Bare-Metal*): S'instal·la directament sobre el maquinari físic de la màquina (sense cap sistema operatiu per sota). És el model utilitzat en el núvol professional (ex: VMware ESXi, KVM, Hyper-V) perquè ofereix el màxim rendiment i eficiència.
- Tipus 2 (Allotjat o *Hosted*): S'executa sobre un sistema operatiu ja existent (com Windows o Linux). És el que s'utilitza a nivell d'usuari (ex: VirtualBox).



5.3. L'aprovisionament de recursos virtuals

A diferència de la virtualització clàssica de laboratori, l'aprovisionament en el núvol és automàtic gràcies a una capa superior de programari anomenada orquestrador (com l'arquitectura interna d'AWS o OpenStack).

- Abstracció: L'orquestrador mesura el maquinari disponible a tot el centre de dades.

- **Assignació dinàmica:** Quan un usuari demana un servidor web amb 2 vCPUs i 4 GB de RAM, l'orquestrador busca quin servidor físic té aquest espai lliure, ordena a l'hipervisor que reservi aquests recursos exactes i aixeca la màquina virtual en segons.
- **Aïllament (*Multitenancy*):** L'hipervisor garanteix que cap client (inquilí) pugui accedir a la memòria ni a les dades d'un altre client, encara que comparteixin el mateix processador físic.

5.4. Preparació de càrregues de treball (*Workload Provisioning*)

Una càrrega de treball (*workload*) és qualsevol aplicació, servei o procés que consumeix recursos de computació (per exemple, un e-commerce, una base de dades o un algorisme d'intel·ligència artificial). Per preparar-les correctament en el núvol, cal seguir tres fases:

1. **Definició del Perfil de Consum:** Escollir el tipus d'instància virtual segons el comportament de l'aplicació. Els proveïdors ofereixen màquines optimitzades per a computació (molta CPU), optimitzades per a memòria (molta RAM) o d'ús general.
2. **Estandardització via Imatges (Templates/AMIs):** La càrrega de treball no s'instal·la a mà. Es crea una imatge base del disc (que inclou el sistema operatiu configurat i el codi de l'aplicació) perquè l'orquestrador pugui clonar-la i llançar noves instàncies de manera idèntica i automatitzada.
3. **Contenidorització (Alternativa moderna):** Actualment, moltes càrregues de treball es preparen utilitzant contenidors (com Docker) en comptes de màquines virtuals completes. Els contenidors comparteixen el mateix sistema operatiu de la màquina host, fent-los molt més lleugers, ràpids d'activar i eficients en l'ús dels recursos del núvol.

6. Caracterització de recursos en el núvol

La caracterització dels recursos consisteix a entendre com els components físics s'abstractuen en elements lògics que els administradors poden configurar des d'un panell de control. Aquests recursos es divideixen en quatre grans pilars integrats per capes de programari base.

6.1. Gestió i tipologies de servidors virtuals

Els servidors virtuals (anomenats *instàncies* o *màquines virtuals*) no són genèrics; els proveïdors els classifiquen en famílies segons la relació entre CPU, RAM i capacitat de disc per optimitzar el cost i el rendiment:

- **Ús general:** Equilibri entre memòria i potència de càlcul. Són ideals per a servidors web de trànsit moderat, entorns de desenvolupament i repositoris de codi.
- **Optimitzats per a computació:** Proporció elevada de vCPUs en relació amb la RAM. S'utilitzen per a processament de dades en lots (batch), codificació de vídeo i servidors web d'alt rendiment.
- **Optimitzats per a memòria:** Gran quantitat de RAM per cada vCPU. Dissenyats per a bases de dades de gran tamany o sistemes de memòria cau en memòria (com *Redis* o *SAP HANA*).
- **Computació accelerada:** Inclouen targetes gràfiques de maquinari (**GPUs**) o xips dedicats (TPUs). Són imprescindibles per a l'entrenament de models d'Intel·ligència Artificial, *Machine Learning* i renderització 3D.

6.2. Xarxes virtuals: L'esquelet de la infraestructura

La xarxa en el núvol està completament definida per programari (**SDN - Software Defined Networking**). Permet crear entorns totalment aïllats per a cada client:

- **VPC (Virtual Private Cloud):** És una xarxa privada virtual, lògicament aïllada de la resta del núvol del proveïdor. L'empresa té el control total sobre el rang d'adreces IP privades, l'encaminament i les passarel·les d'Internet.
- **Subxarxes (Subnets):** Divisions internes d'una VPC que permeten organitzar i seguitzar els recursos:
- **Subxarxa Pública:** Té accés directe a Internet. S'hi col·loquen els recursos que han de rebre trànsit extern, com els balancejadors de càrrega o els servidors web frontals.
- **Subxarxa Privada:** No és accessible des d'Internet. S'hi ubiquen els elements crítics com les bases de dades o la lògica de negoci, protegint-los d'atacs externs.

6.3. Tipologies d'emmagatzematge en el núvol

L'emmagatzematge es classifica segons la forma en què s'accedeix a les dades i l'estructura que utilitzen:

- **Emmagatzematge de blocs (Block Storage):** Funciona com un disc dur físic tradicional connectat directament al servidor (ex: AWS EBS). Les dades es divideixen en blocs de mida fixa. És idoni per a sistemes operatius i bases de dades a causa de la seva **baixa latència** i alt rendiment de lectura/escriptura (IOPS).
- **Emmagatzematge d'objectes (Object Storage):** Les dades es guarden com a objectes autònoms que contenen el fitxer, metadades detallades i un identificador únic (ex: AWS S3). No utilitza una estructura de carpetes, sinó un espai pla. És pràcticament **il·limitat i molt econòmic**, ideal per a còpies de seguretat, imatges, vídeos i grans volums de dades no estructurades.
- **Emmagatzematge de fitxers (File Storage):** Un sistema de fitxers compartit en xarxa basat en protocols tradicionals com NFS o SMB (ex: AWS EFS). Permet que **múltiples servidors virtuals llegeixin i escriguin fitxers alhora** sobre el mateix volum de forma concurrent.

Resum de tipus d'emmagatzematge

Tipus	Cas d'ús principal	Característica clau
Blocs	Instal·lar el Sistema Operatiu i Bases de dades relacionals.	Màxima velocitat i baixa latència.
Objectes	Allotjar fitxers estàtics de webs, vídeos i Backups.	Creixement il·limitat i cost molt baix.
Fitxers	Compartir dades entre diversos servidors en paral·lel.	Accés concurrent per a architectures multi-servidor.

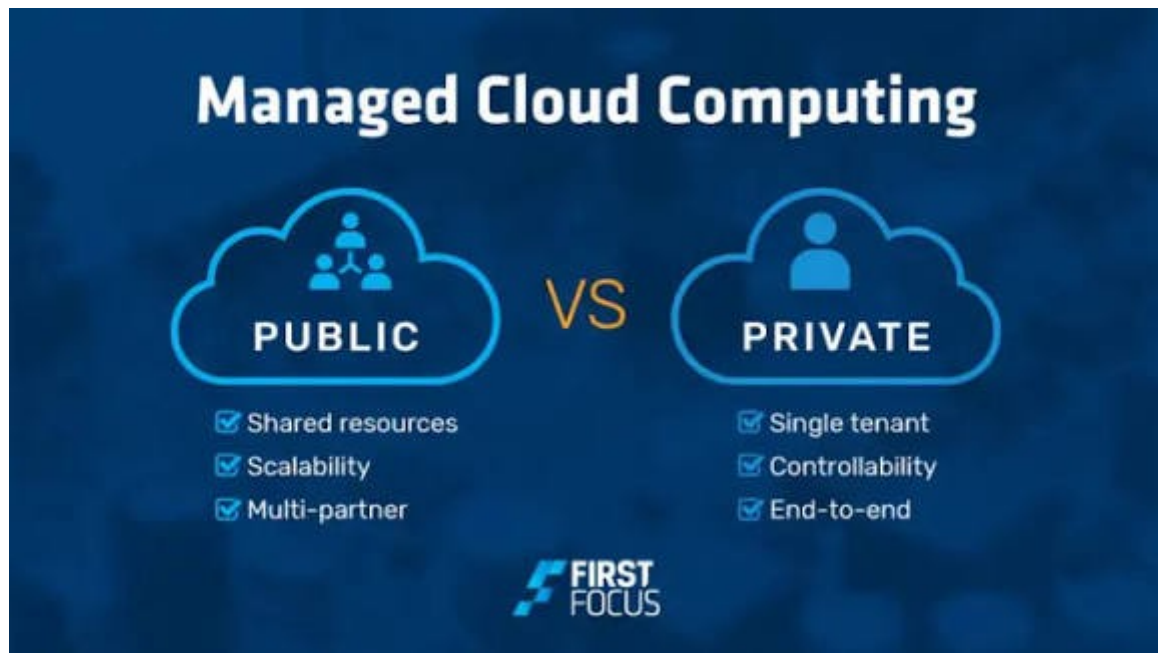
6.4. Capes de programari base

Perquè l'usuari final pugui desplegar aplicacions sobre aquests recursos, interactuen diferents capes de programari de base:

1. **Sistemes Operatius Convidats (Guest OS):** Les distribucions de Linux (Ubuntu, RedHat) o Windows Server adaptades per al núvol, que inclouen controladors específics per interactuar amb l'hipervisor de forma eficient.
2. **Controladors de xarxa i disc virtuals (Drivers para-virtualitzats):** Programari que optimitza la comunicació entre el Sistema Operatiu i el maquinari real per reduir la penalització de rendiment de la virtualització.
3. **Agents del proveïdor (Cloud-init / SSM):** Xicotets serveis que s'executen en iniciar la màquina virtual i permeten automatitzar la configuració inicial (com injectar claus SSH, executar scripts de configuració o recollir mètriques de rendiment).

--ooOOoo--

Tema 3: Models d'implementació i desplegament en el núvol



1. El model de Núvol Públic
2. El model de Núvol Privat
3. El model de Núvol Híbrid
4. Criteris de selecció d'entorns
5. Entorns Multinúvol (Multicloud)
6. Administració i governança avançada

1. El model de Núvol Públic

El model de Núvol Públic és la base del mercat de la computació en el núvol. Representa el canvi tecnològic més democratitzador del sector, ja que posa infraestructures a escala global a l'abast de qualsevol organització, des d'una start-up d'un sol fundador fins a una gran multinacional.

1.1. Estructura de propietat i control

En el núvol públic, tota la infraestructura física és propietat d'un **proveïdor de serveis extern** (conegut com a *Hyperscaler*), que s'encarrega d'adquirir, construir, allotjar i mantenir els equips en els seus propis centres de dades globals.

- **Els líders del mercat:** Els proveïdors de referència a nivell mundial són **Amazon Web Services (AWS)**, **Microsoft Azure** i **Google Cloud Platform (GCP)**.
- **Accés a recursos:** Els usuaris o empreses clients no tenen permís per entrar físicament als centres de dades ni veure el maquinari real. Tota la gestió es realitza de forma remota a través d'Internet mitjançant consoles web, línies de comandes (CLI) o interfícies de programació d'aplicacions (APIs).

1.2. Arquitectura i organització geogràfica

L'estructura del núvol públic es dissenya de manera jeràrquica per garantir la disponibilitat a tot el món:

- **Regions:** Ubicacions geogràfiques reals a tot el planeta (per exemple, la regió d'AWS a Espanya o la de Microsoft a Irlanda). Cada regió està completament aïllada de les altres de manera política i geogràfica.
- **Zones de Disponibilitat (AZ):** Dins de cada regió hi ha múltiples zones de disponibilitat (mínim tres). Una AZ està formada per un o diversos centres de dades físics independents, separats per uns quants quilòmetres, amb línies elèctriques i de xarxa redundants però connectats entre si per fibra òptica d'ultra alta velocitat.

1.3. El principi del multiinquilí (Multi-tenancy)

El motor econòmic i operatiu del núvol públic és el **model multiinquilí (multi-tenant)**. Aquest principi determina que **múltiples clients (inquilins) comparteixen els mateixos recursos físics subjacents** de manera simultània.

- **La metàfora de l'edifici d'oficines:** El núvol públic funciona com un bloc de pisos o oficines. El proveïdor (propietari de l'edifici) manté l'estructura, l'ascensor i la seguretat de la porteria. Cada empresa lloga una oficina (un entorn virtual). Totes les oficines comparteixen els mateixos fonaments i canonades (maquinari, xarxa i processadors), però les portes estan tancades i cap empresa pot veure el que fa la del costat.
- **Aïllament lògic estricte:** Els clients estan completament separats a nivell de programari (lògicament). L'hipervisor s'encarrega de garantir que el trànsit de xarxa, les dades de la

memòria RAM i la informació emmagatzemada en disc d'un inquilí estiguen absolutament protegits de la resta d'inquilins.

- **Economia d'escala:** Compartir el maquinari permet als proveïdors optimitzar l'ús dels servidors físics al màxim. Això redueix dràsticament els costos d'explotació, repercutint en uns preus molt més econòmics per als clients individuals en comparació amb el cost de muntar un centre de dades propi.

1.4. Resum conceptual del Núvol Públic

Característica	Detall en el Núvol Públic
Inversió inicial	Zero. No cal comprar maquinari ni pagar per la instal·lació física.
Responsabilitat de manteniment	Totalment del proveïdor (AWS, Azure, GCP, etc.).
Seguretat de les dades	Garantida per programari. Aïllament criptogràfic i lògic entre inquilins.
Elasticitat	Gairebé infinita. Capacitat immediata de créixer gràcies a la immensitat de la infraestructura del proveïdor.

2. El model de Núvol Privat

El **Núvol Privat** representa l'evolució conceptual del centre de dades tradicional cap al model cloud. A diferència del núvol públic, on els recursos es comparteixen de forma massiva, el núvol privat es defineix per la **exclusivitat**, oferint l'agilitat i l'automatització pròpies del núvol, però dins d'un entorn dedicat a un únic client.

2.1. Arquitectura dedicada i propietat exclusiva dels recursos

La clau d'un núvol privat és el principi de **monoinquilí (single-tenancy)**. Tota la pila tecnològica està reservada per a una sola organització.

- **Sense compartició de maquinari:** A diferència del núvol públic, cap altra empresa comparteix els mateixos processadors, memòria o xarxa física. Això elimina per complet riscos associats com el del "*veí sorollós*" (on un altre client consumeix massa recursos i alenteix el teu servidor).
- **Control total de la infraestructura:** L'empresa té la capacitat de triar exactament les especificacions del maquinari, els sistemes de virtualització (com *VMware Cloud Foundation* o *OpenStack*) i definir les polítiques exactes d'actualització i configuració de l'hipervisor.

2.2. Seguretat perimetral avançada i governança

El núvol privat destaca en entorns on el control total de la informació és un requisit crític del negoci:

- **Aïllament físic i lògic:** En estar desconnectat de l'abast públic o tindre un control absolut sobre les passarel·les d'entrada, la **seguretat perimetral** es pot dissenyar a mida amb tallafocs (*firewalls*) físics i sistemes de detecció d'intrusions (IDS/IPS) d'última generació.
- **Sobirania de les dades i compliment normatiu:** Per a indústries altament regulades (banca, sector sanitari o defensa), aquest model garanteix el compliment estricte de lleis

de privacitat nacionals o internacionals, assegurant que les dades mai surten d'un radi de control físic determinat.

2.3. Casos d'ús i modalitats de desplegament

El núvol privat no s'ha d'ubicar obligatòriament a l'oficina central. Hi ha dues modalitats principals segons la seva localització:

- **Núvol Privat On-Premise (Centres de dades propis):** L'empresa construeix i gestiona el núvol a les seves pròpies instal·lacions.
- *Cas d'ús:* Grans corporacions bancàries o entitats governamentals que disposen de personal altament especialitzat i edificis blindats per allotjar els servidors, on necessiten un control absolut des del cable de fibra fins a l'aplicació final.
- **Núvol Privat Allotjat (Hosted / Managed Private Cloud):** La infraestructura física es troba en el centre de dades d'un **proveïdor extern** (com *Equinix*, *IBM* o proveïdors de colocalització), però aquest proveïdor reserva racks i servidors físics concrets exclusivament per a eixe client. La gestió del maquinari es pot delegar en el proveïdor.
- *Cas d'ús:* Una empresa mitjana que requereix el màxim aïllament per als seus sistemes crítics de gestió (ERP), però no vol assumir els costos de llogar un edifici, contractar seguretat física o gestionar els sistemes de refrigeració i electricitat.

2.4. Resum conceptual del Núvol Privat

Avantatges Clau	Inconvenients i Reptes
Control absolut: Personalització total del maquinari i programari.	Alt cost inicial: Retorn al model CapEx (cal comprar o finançar els equips).
Màxima privacitat: Aïllament total de les dades i el trànsit de xarxa.	Escalabilitat limitada: Si es necessiten més servidors, cal comprar-los i esperar que arribin.
Rendiment predictable: Sense interferències de tercers clients.	Manteniment intern: L'equip de TI torna a ser responsable del cicle de vida del maquinari.

3. El model de Núvol Híbrid

El **Núvol Híbrid** és el model de desplegament més adoptat per les mitjanes i grans empreses. En lloc de triar un camí únic, el núvol híbrid combina elements del **núvol públic** i d'entorns **privats o locals (On-Premise)**, permetent que funcionin com una infraestructura única, automatitzada i cohesionada.

3.1. La clau de la connectivitat: Ponts d'unió

Perquè un entorn híbrid funcione, cal establir connexions segures, d'alta velocitat i de baixa latència entre el centre de dades privat i les regions del proveïdor públic (AWS, Azure, GCP). S'utilitzen principalment dues tecnologies:

- **Xarxes Privades Virtuals (VPN Site-to-Site):** Connecten ambdós entorns a través d'Internet de forma xifrada (mitjançant protocols com IPsec).

- **Característica:** És una solució econòmica i ràpida de configurar, ideal per a fluxos de dades moderats.
- **Connexions Dedicades Directes:** Enllaços de fibra òptica físics i privats que esquiven l'Internet públic (ex: *AWS Direct Connect*, *Azure ExpressRoute*).
- **Característica:** Ofereixen una amplada de banda massiva (fins a 100 Gbps), latències extremadament baixes i un rendiment altament predictable i segur.

3.2. Fluxos de dades i orquestració entre entorns

La connectivitat permet dissenyar fluxos de treball dinàmics on les dades i les aplicacions es mouen d'un entorn a l'altre segons les necessitats operatives:

- **L'extensió del núvol (*Cloud Bursting*):** L'aplicació s'executa habitualment al núvol privat (on els costos són fixos). Si hi ha un pic sobtat de demanda (per exemple, una campanya de màrqueting), l'arquitectura es desborda de forma automàtica cap al núvol públic per absorbir el trànsit excedent, evitant la caiguda del servei. Quan el pic passa, els recursos públics s'apaguen.
- **Separació per capes d'aplicació:** Permet col·locar cada component on és més eficient o segur.
- **Capa de dades:** Les bases de dades crítiques o regulades es queden al núvol privat (per seguretat o sobirania).
- **Capa de processament/front-end:** Els servidors web o les eines d'analítica de dades s'executen al núvol públic per aprofitar el seu gran abast i potència de càlcul.
- **Còpies de seguretat i continuïtat de negoci (DRaaS):** Els sistemes principals funcionen en l'entorn local, però utilitzen el núvol públic com a destinació de les rèpliques i còpies de seguretat, reduint radicalment els costos de tenir un segon centre de dades físic.

3.3. El repte de l'homogeneïtat tècnica

Perquè les aplicacions es puguin moure entre el núvol privat i el públic sense haver de reescriure el codi, la indústria utilitza **capes d'abstracció unificades**:

- **Contenidors i Kubernetes:** Permeten empaquetar l'aplicació de manera que s'executi exactament igual en un servidor local que en una instància d'AWS.
- **Plataformes híbrides natives:** Els propis *hyperscalers* ofereixen programari per gestionar el teu centre de dades local com si fos part del seu núvol (ex: *AWS Outposts*, *Azure Arc*, *Google Anthos*).

3.4 Casos d'ús d'Arquitectura Híbrida

Objectiu del Negoci	Rol del Núvol Privat	Rol del Núvol Públic
Compliment i Elasticitat	Allotja les dades bancàries i personals dels clients.	Executa les aplicacions mòbils de cara a l'usuari.
Migració Progressiva	Manté els sistemes antics (<i>legacy</i>) mentre es preparen per al canvi.	Rep les noves aplicacions desenvolupades de zero.
Intel·ligència de Negoci	Guarda els fitxers històrics de l'empresa.	Processa grans volums de dades amb algorismes de <i>Machine Learning</i> .

4. Criteris de selecció d'entorns

Per tal de prendre una decisió estratègica sobre on ubicar els sistemes de l'empresa, cal fer una **avaluació comparativa ponderada**. La tria d'un entorn (Públic o Privat) no s'ha de basar en quina tecnologia és millor en termes absoluts, sinó en quin model respon millor als requisits tècnics, econòmics i legals del projecte.

A continuació, es desenvolupen els cinc criteris fonamentals de selecció:

1. Costos (CapEx vs. OpEx)

- **Núvol Públic:** Funciona sota un model de despeses operatives (**OpEx**). És ideal per a empreses que busquen una inversió inicial de **zero euros** i prefereixen la flexibilitat del pagament per ús. No obstant això, si una infraestructura funciona al 100% de la seva capacitat les 24 hores del dia de forma sostinguda durant anys, el núvol públic pot arribar a ser més car que el privat a llarg termini.
- **Núvol Privat:** Requereix una inversió de capital inicial (**CapEx**) molt elevada per adquirir el maquinari i les llicències. Tot i així, per a grans corporacions amb càrregues de treball constants, predictibles i massives, el cost per unitat de computació pot acabar sent inferior i més fàcil de pressupostar a llarg termini (costos fixos).

2. Seguretat digital

- **Núvol Públic:** Els grans proveïdors (AWS, Azure, GCP) inverteixen milers de milions en ciberseguretat, disposant d'equips d'experts i sistemes de detecció d'amenaces que cap empresa mitjana podria finançar per si mateixa. Tot i això, en ser un entorn compartit (*multi-tenant*), el **risc d'errors de configuració per part de l'usuari** (com deixar un repositori de dades obert a Internet) és una de les principals vulnerabilitats.
- **Núvol Privat:** Ofereix el màxim control de la seguretat perimetral. L'aïllament físic i el model monoinquilí (*single-tenant*) redueixen dràsticament la superfície d'atac externa. El risc principal es desplaça a l'interior: depèn totalment de la capacitat i formació de l'equip de seguretat intern de la mateixa empresa.

3. Compliment normatiu i Sobirania de dades

- **Núvol Públic:** Tot i que els *hyperscalers* disposen de certificacions internacionals estrictes (com l'ISO 27001 o l'Esquema Nacional de Seguretat a Espanya), certes normatives governamentals o sectors hiperregulats prohibits o limiten l'allotjament de dades altament confidencials en servidors de tercers, especialment si estan gestionats per empreses estrangeres.
- **Núvol Privat:** És l'opció guanyadora quan es requereix un control absolut sobre la **sobirania de les dades**. Garanteix exactament en quin servidor, habitació i país es troba la informació, facilitant l'aprovació d'auditories legals complexes en sectors com la salut, la defensa o la banca central.

4. Control i personalització tècnica

- **Núvol Públic:** L'usuari té un control complet sobre les seves aplicacions i dades, però **zero control sobre el maquinari i l'hipervisor**. Les empreses s'han d'adaptar a les configuracions, catàlegs de màquines i calendaris d'actualització que dicta el proveïdor.
- **Núvol Privat:** Ofereix una llibertat absoluta de personalització. L'empresa pot escollir arquitectures de xarxa a mida, maquinari específic de nínxol (com targetes acceleradores molt concretes o sistemes d'emmagatzematge singulars) i decidir quan es fan les parades de manteniment tècnic sense dependre de tercers.

5. Escalabilitat i Velocitat (Time-to-Market)

- **Núvol Públic:** L'escalabilitat és el seu punt fort. És **pràcticament il·limitada i immediata**. Una empresa pot passar d'1 a 500 servidors en qüestió de minuts per respondre a un pic de trànsit global, fent-lo ideal per a negocis digitals amb demandes canviants o en fase de creixement ràpid.
- **Núvol Privat:** L'escalabilitat està **limitada per la capacitat física disponible** a les cabines de servidors. Si el sistema arriba al 100% de la seva capacitat i es necessita créixer, el procés d'adquisició, tramesa logística, instal·lació i configuració de nou maquinari pot trigar setmanes o mesos.

Taula de decisió directa

Criteri d'avaluació	Triar Núvol Públic si...	Triar Núvol Privat si...
Perfil Financer	Prefereixes costos operatius variables (OpEx) sense inversió inicial.	Disposes de pressupost inicial (CapEx) i busques costos fixos predictibles.
Comportament de la Demanda	Tens pics de trànsit impredecibles o projectes estacionaris.	La teva càrrega de treball és estable, constant i molt coneguda.
Nivell de Regulació Legal	El teu sector permet l'ús de tercers amb certificacions estàndard.	Estàs subjecte a lleis estrictes de sobirania de dades o secret professional.
Requisits de Maquinari	T'adaptes a les mides i tipologies de servidors estàndard del mercat.	Necessites connectar maquinari especialitzat o sistemes molt antics (<i>legacy</i>).

5. Entorns Multinúvol (Multicloud)

L'estratègia Multinúvol (Multicloud) s'ha convertit en el model de referència per a les organitzacions que busquen la màxima resiliència i independència tecnològica. Consistix a fer servir simultàniament serveis de dos o més proveïdors de núvol públic diferents (per exemple, AWS i Microsoft Azure) per a allotjar la infraestructura d'una mateixa empresa.

A continuació, es desenvolupen les claus tècniques i operatives d'aquest model:

5.1. Diferències clau: Multicloud vs. Núvol Híbrid

Tot i que ambdós conceptes impliquen treballar amb més d'un entorn de núvol, la seva naturalesa conceptual i els seus objectius són diferents:

- **Núvol Híbrid:** Se centra en la **interconnexió**. Uneix un entorn de núvol públic (ex. AWS) amb un entorn local o privat (On-Premise). L'objectiu és que funcionin com una sola xarxa unificada per a estendre les capacitats locals.
- **Multicloud:** Se centra en la **diversificació**. Utilitza múltiples plataformes de núvol públic de diferents marques (ex. combinar AWS, Azure i GCP). No té per què incloure un centre de dades privat, tot i que un entorn pot ser híbrid i multinúvol alhora (*Hybrid Multicloud*).

5.2. Evitar el bloqueig del proveïdor (*Vendor Lock-in*)

El *vendor lock-in* és el risc de dependre tant dels serveis exclusius d'un proveïdor que el cost, el temps i l'esforç de migrar a un altre es tornen prohibitius. L'enfocament multinúvol és la millor estratègia per a mitigar aquest problema:

- **Independència comercial:** Evita quedar atrapat per pujades de preus unilaterals del proveïdor o per canvis en els seus termes de servei.
- **Poder de negociació:** Permet a les empreses negociar millors tarifes en tenir la capacitat tècnica demostrada de moure les seves operacions a la competència en qualsevol moment.
- **Optimització de característiques:** Permet triar "el millor de cada casa" (*Best-of-Breed*). Per exemple, utilitzar AWS per a l'emmagatzematge massiu, Azure per a la integració amb aplicacions de Windows, i Google Cloud per a les eines avançades de Big Data i Intel·ligència Artificial.

5.3. Requisits tècnics per a una estratègia Multinúvol eficient

Operar en multinúvol afegeix una gran complexitat. Per fer-ho funcionar sense duplicar el personal ni els errors, calen tres pilars tècnics:

- **Abstracció mitjançant Contenedors (Kubernetes):** És imprescindible dissenyar aplicacions agnòstiques. En lloc de programar directament per a les eines natives d'un núvol, s'utilitza Kubernetes com a capa intermèdia. Així, el mateix codi funciona exactament igual en qualsevol proveïdor.
- **Infraestructura com a Codi (IaC) Unificada:** Eines com **Terraform** o **Pulumi** permeten definir la topologia de les xarxes i servidors de tots els proveïdors utilitzant un sol llenguatge de programació, automatitzant el desplegament en qualsevol entorn de forma idèntica.
- **Plataformes de Gestió Centralitzada (FinOps i Seguretat):** Cal utilitzar programari de tercers que aglutini la monitorització de costos, l'estat dels sistemes i les polítiques de ciberseguretat en un únic quadre de comandament centralitzat per a tots els núvols.

5.4. Escenaris pràctics d'ús

- **Resiliència extrema i Alta Disponibilitat:** Si una regió sencera d'AWS cau o pateix un atac global, el trànsit d'usuaris es redirigeix de forma automatitzada cap a Microsoft Azure mitjançant sistemes de DNS intel·ligents, garantint que l'aplicació web mai quedi fora de servei.

- **Distribució per Geografies i Latència:** Si una empresa opera a nivell mundial, pot utilitzar el proveïdor que tingui el centre de dades més proper i amb menor latència a cada continent (per exemple, utilitzar *Alibaba Cloud* per a les operacions a Àsia i *GCP* per a Amèrica).
- **Migració i Gestió del Risc:** Dividir les línies de negoci crítiques de l'empresa. Per exemple, col·locar el sistema de facturació en un núvol i l'entorn de desenvolupament i proves en un altre proveïdor més econòmic.

5.5. Taula: Híbrid vs. Multinúvol

Característica	Núvol Híbrid	Entorn Multinúvol
Composició típica	1 Núvol Públic + 1 Entorn Local (On-Premise).	2 o més Núvols Públics diferents (AWS, Azure...).
Objectiu Principal	Aprofitar la inversió local i estendre-la al núvol.	Evitar el monopoli d'un proveïdor i augmentar la resiliència.
Complexitat de Gestió	Mitjana (es connecten dos mons).	Alta (cal dominar les tecnologies de diferents marques).

6. Administració i governança avançada

Gestionar un ecosistema tecnològic que combina centres de dades propis amb múltiples proveïdors públics (com AWS, Azure i GCP) introdueix una gran complexitat operativa. L'administració i governança avançada consisteix a establir el marc de control, visibilitat i automatització necessari perquè aquesta infraestructura fragmentada funcione com un sistema únic, segur i eficient.

6.1. Estratègies de gestió unificades

El repte principal dels entorns complexos és evitar les operacions en "silos" (on cada equip gestiona un núvol amb eines diferents). S'apliquen tres estratègies clau:

- **Consoles de gestió híbrides (Single Pane of Glass):** Ús de plataformes d'orquestració que actuen com una capa superior sobre tots els entorns. Exemples com **Azure Arc**, **Google Anthos** o **AWS Outposts** permeten estendre els serveis de control nadius cap a altres núvols o servidors locals.
- **Cultura FinOps (Cloud Financial Operations):** Pràctica de gestió financera que uneix els equips de tecnologia, finances i negoci. En entorns multinúvol, és imprescindible utilitzar eines d'analítica de costos (com *Cloudability* o *Harness*) per a consolidar la facturació, detectar recursos infrautilitzats i preveure despeses de forma centralitzada.
- **Infraestructura com a Codi (IaC) i GitOps:** Tot el desplegament i modificació de la xarxa o els servidors es definix mitjançant fitxers de text (amb eines com **Terraform**). Aquests fitxers es guarden en un repositori de codi (com Git), garantint que qualsevol canvi en qualsevol núvol siga auditable, repetible i automàtic.

6.2. Monitorització centralitzada i observabilitat

En entorns híbrids, no n'hi ha prou amb saber si un servidor està encés o apagat; cal una **observabilitat completa** (mètriques, registres de sistema o *logs*, i traces d'aplicacions) per a detectar problemes abans que afecten l'usuari.

- **Agregació de registres (Log Aggregation):** Totes les màquines virtuals, bases de dades i tallafocs de tots els núvols envien els seus registres a un repositori centralitzat (com *Elasticsearch*, *Splunk* o *Datadog*). Això permet correlacionar esdeveniments (per exemple, comprovar si una fallada en una base de dades local està afectant l'aplicació allotjada a AWS).
- **Intel·ligència Artificial per a Operacions (AIOps):** Ús d'algorismes de *Machine Learning* per a analitzar milers de milions de mètriques en temps real, detectar anomalies de rendiment automàticament i llançar alertes intel·ligents, evitant la saturació d'avisos repetitius per als enginyers.

6.3. Polítiques de compliment i seguretat automatitzada

Garantir la seguretat i la legalitat en entorns complexos requereix substituir les auditories manuals per polítiques automatitzades en temps real (Policy-as-Code):

- **Control d'identitats unificat (Identity Federation):** Implementació d'un sistema de *Single Sign-On* (SSO) centralitzat (com *Active Directory* o *Okta*). Si un empleat deixa l'empresa, es revoquen els seus accessos a tots els núvols públics i privats de forma immediata des d'un únic punt.
- **Seguretat i Compliment Continu (CSPM):** Eines de Gestió de la Postura de Seguretat al Núvol (*Cloud Security Posture Management*) que escanegen constantment tots els entorns per a comprovar si compleixen amb les normatives (com la RGPD o l'ISO 27001). Per exemple, si un desenvolupador obri per error un port de xarxa prohibit a GCP, el sistema de governança el detecta, llança una alerta i pot tancar el port automàticament.
- **Etiquetatge obligatori (Tagging Policies):** Polítiques automatitzades que impedeixen crear cap recurs (servidor, disc, xarxa) si no inclou etiquetes que identifiquen qui n'és el propietari, a quin departament pertany i quin és el seu propòsit. Això és vital per a poder assignar els costos correctament en el model FinOps.

6.4. Resum dels pilars de la governança avançada

Pilar	Objectiu principal	Eina / Tecnologia comuna
Control Operatiu	Gestionar tots els núvols des d'una única interfície.	Azure Arc, Google Anthos, Terraform.
Visibilitat	Detectar errades correlacionant dades de tot l'ecosistema.	Datadog, Dynatrace, Splunk.
Control Financer	Evitar el malbaratament de pressupost entre proveïdors.	Pràctiques FinOps, eines de Cloud Cost Optimization.
Seguretat i Llei	Garantir que cap recurs incompleix les normes de l'empresa.	Policy-as-Code (Open Policy Agent), sistemes CSPM.

--ooOOoo--

Tema 4: Models de servei en l'arquitectura de núvol (XaaS)



1. Infraestructura com a Servei (IaaS)
2. Plataforma com a Servei (PaaS)
3. Programari com a Servei (SaaS)
4. Arquitectura sense servidor (Serverless)
5. Matriu de responsabilitat compartida

1. Infraestructura com a Servei (IaaS)

El model d'**Infraestructura com a Servei (IaaS)** representa la capa fonamental i més flexible de la computació en el núvol. En aquest model, el proveïdor de Cloud lloga els recursos de maquinari pur en format virtual, actuant com una alternativa directa a tenir un centre de dades físic pròpia.

A continuació, es desenvolupa l'estructura, el funcionament i el model de control de la IaaS:

1.1. Estructura i components del model

En un entorn IaaS, el proveïdor de serveis (com AWS, Microsoft Azure o Google Cloud) és el propietari i responsable de tota la **infraestructura física subjacent**. L'usuari adquireix l'accés a components virtualitzats agrupats en quatre categories essencials:

- **Recursos de computació:** Processadors virtuals (vCPUs) i memòria RAM.
- **Emmagatzematge:** Discos virtuals (de blocs) associats o independents a les màquines.
- **Xarxa:** Targetes d'interfície de xarxa virtuals, adreces IP públiques/privades i encaminadors.
- **Capa de virtualització:** L'hipervisor controlat pel proveïdor que segmenta el maquinari real.

1.2. Funcionament operatiu

El funcionament de la IaaS es basa en el programari d'orquestració automatitzat. L'usuari no ha d'esperar que un tècnic instal·li físicament un servidor:

1. **Sol·licitud de recurs:** A través de la consola web, d'una API o de codi (IaC), l'usuari demana una instància concreta (per exemple, un servidor amb 4 vCPUs i 16 GB de RAM).
2. **Assignació immediata:** L'orquestrador del proveïdor localitza un servidor físic amb capacitat lliure, reserva els recursos a través de l'hipervisor i clona una imatge de disc inicial de forma gairebé instantània.
3. **Accés remot:** En qüestió de segons, la màquina es posa en marxa i l'usuari en rep l'accés complet mitjançant protocols de connexió segura com **SSH** (en Linux) o **RDP** (en Windows).

1.3. Control total de l'usuari: El sistema operatiu i la xarxa

La principal característica de la IaaS és el seu **alt nivell de control i personalització**, ja que el proveïdor s'atura a la capa de virtualització. A partir d'aquí, l'usuari té el control absolut de:

- **El Sistema Operatiu (SO):** L'usuari té permisos d'administrador total (*root* o *administrator*). Pot triar quina distribució exacta instal·lar (Ubuntu, RedHat, Windows Server), configurar el nucli (*kernel*) del sistema i aplicar els pedaços de seguretat quan ho consideri oportun.

- **Configuració de xarxa interna:** L'empresa dissenya les taules d'encaminament, obre o tanca els ports mitjançant tallafocs virtuals (*Security Groups*) i decideix quines màquines tenen accés directe a Internet i quines queden completament aïllades.
- **Programari de base:** L'usuari és l'encarregat d'instal·lar i configurar els motors de bases de dades (com MySQL o Oracle), els servidors web (com Nginx o Apache) i els entorns d'execució del codi.

1.4. Exemples de referència al mercat

Tots els grans proveïdors disposen d'un servei IaaS estrella que serveix de base per a la resta de les seves arquitectures:

- **Amazon Web Services (AWS): Amazon EC2** (*Elastic Compute Cloud*).
- **Microsoft Azure: Azure Virtual Machines** (Azure VMs).
- **Google Cloud Platform (GCP): Google Compute Engine** (GCE).

1.5. El Model de Responsabilitat Compartida en IaaS

La IaaS és el model de núvol on l'usuari assumeix la major part de la responsabilitat tècnica:

El Proveïdor gestiona i protegeix	L'Usuari gestiona i protegeix
Seguretat física del centre de dades (racks, energia, accés).	El Sistema Operatiu (actualitzacions i pedaços).
Manteniment del maquinari real (servidors, cablatge, xarxa física).	Controls d'accés i seguretat de la xarxa virtual (tallafocs).
Funcionament correcte de l'hipervisor de virtualització.	Instal·lació de programari, codi i protecció de les dades.

2. Plataforma com a Servei (PaaS)

El model de **Plataforma com a Servei (PaaS)** representa el següent nivell d'abstracció en el núvol. A diferència de la IaaS, on l'usuari s'ha d'ocupar de tota la configuració tècnica, la PaaS elimina la necessitat de gestionar la infraestructura subjacent per permetre que els equips de desenvolupament es concentrin exclusivament en allò que realment aporta valor: el codi de les seves aplicacions.

2.1. Estructura i abstracció del sistema operatiu

En un entorn PaaS, el proveïdor no només s'encarrega del maquinari físic i de l'hipervisor, sinó que també **assumeix la responsabilitat de tota la pila de programari base**:

- **El Sistema Operatiu (SO):** L'usuari no té accés de tipus *root* ni administrador al sistema operatiu. El proveïdor gestiona les actualitzacions, l'aplicació de pedaços de seguretat i el rendiment del SO de forma totalment transparent per al client.

- **El programari d'execució (Runtime):** La plataforma ja inclou els entorns configurats per a executar llenguatges de programació específics (com *Node.js*, *Python*, *Java*, *PHP*, *.NET* o *Ruby*).
- **Serveis intermedis (Middleware):** La plataforma proveeix i gestiona de forma automàtica els servidors web (com Nginx o Tomcat) i les connexions amb les bases de dades.

2.2. Funcionament operatiu: Enfocament exclusiu en el codi

El flux de treball en una PaaS és extremadament senzill per als desenvolupadors, ja que es resumeix en el concepte de "**pujar codi i executar**":

1. **Desenvolupament local:** El programador escriu l'aplicació web al seu ordinador utilitzant el seu llenguatge preferit.
2. **Desplegament automatitzat:** En lloc de configurar servidors web per SSH, el desenvolupador envia el codi a la PaaS mitjançant una comanda (ex: `git push heroku main`) o connectant el repositori de GitHub.
3. **Orquestració automàtica:** La PaaS detecta quin llenguatge utilitza el codi, aixeca els contenidors o servidors virtuals necessaris en segons, instal·la les dependències del projecte, configura el balancejador de càrrega i assigna una adreça web pública actua immediatament.

2.3. Gestió i elasticitat simplificada

Tot i que l'usuari perd control sobre el sistema operatiu, guanya capacitats avançades de gestió gestionades amb un sol clic:

- **Escalabilitat simplificada:** Si l'aplicació web necessita més potència per un pic de trànsit, no cal redimensionar màquines virtuals. Es pot augmentar el nombre d'instàncies d'execució mitjançant una barra lliscant a la consola web o activant regles d'autoescalat basades en el trànsit.
- **Alta disponibilitat integrada:** El proveïdor s'encarrega que, si la màquina subjacent que executa el codi falla, l'aplicació es mogui automàticament a una altra sense que l'usuari hagi d'intervenir ni dissenyar complexes arquitectures de xarxa.

2.4. Exemples de referència al mercat

- **Heroku:** Un dels pioners històrics de la PaaS, molt popular per la seva senzillesa extrema a l'hora de desplegar aplicacions web amb pocs clics.
- **AWS Elastic Beanstalk:** La solució de PaaS d'Amazon. L'usuari puja el codi i Beanstalk gestiona automàticament el desplegament, el balanç de càrrega i l'autoescalat, utilitzant per sota serveis IaaS d'AWS (com EC2) però mantenint-los ocults per a l'usuari.
- **Altres exemples:** *Google App Engine*, *Azure App Services* i plataformes modernes com *Vercel* o *Render*.

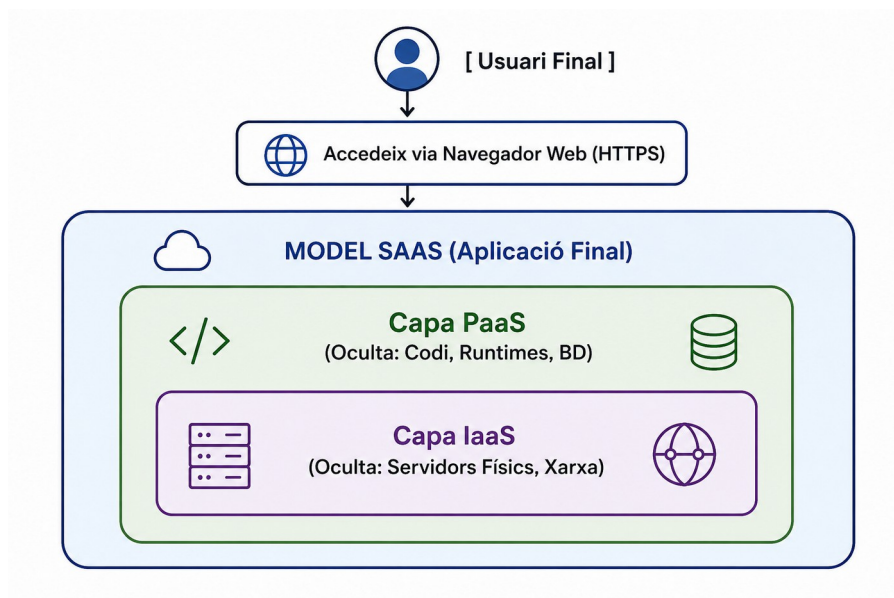
3. Programari com a Servei (SaaS)

El **Programari com a Servei (SaaS)** representa el nivell màxim d'abstracció i comoditat en el model de la computació en el núvol. En aquest escenari, l'empresa o l'usuari final no compra maquinari (IaaS) ni desenvolupa codi (PaaS), sinó que consumeix una aplicació totalment acabada i funcional a través d'Internet, generalment mitjançant un navegador web o una aplicació mòbil.

3.1. Estructura del model: Abstracció tecnològica total

El model SaaS elimina qualsevol barrera tècnica d'entrada. El proveïdor del programari assumeix la responsabilitat absoluta de tota la cadena de valor:

El manteniment operatiu: El proveïdor s'encarrega d'actualitzar el programari per incloure noves funcionalitats, corregir errors (*bugs*), aplicar pedaços de seguretat i garantir que el sistema estigui operatiu les 24 hores del dia.



3.2. Funcionament del model de negoci: Subscripció i accés web

- **Accés instantani i ubicat:** No cal instal·lar ni configurar programari complex a l'ordinador local. Només es requereix una connexió a Internet i unes credencials d'accés (usuari i contrasenya). Les dades es guarden de forma centralitzada al núvol, permetent treballar des de qualsevol lloc i dispositiu.
- **Model comercial (Subscripció / SaaS):** En lloc de comprar una llicència de programari tradicional de per vida (que requeria instal·lar un CD-ROM i actualitzar-lo manualment), es paga una quota periòdica (mensual o anual). Els preus solen basar-se en el nombre d'usuaris (cost per llicència activa) o en el volum de funcionalitats utilitzades.

3.3. Control de l'usuari final: Configuració funcional

En el SaaS, el control tècnic és de zero euros i zero minuts de gestió de servidors. L'únic control que manté l'usuari és a nivell de configuració funcional i de negoci:

- Creació i gestió dels perfils d'usuaris de la seva empresa (rols d'administrador, editor, lector).
- Personalització visual bàsica (afegir el logotip de l'empresa, colors corporatius).
- Configuració de fluxos de treball interns propis de l'aplicació (creació de regles, plantilles de correu, o mètodes de pagament).

3.4. Exemples de referència al mercat

- **Microsoft 365:** Anteriorment comprat com una caixa (Office), avui es consumeix com un servei al núvol que inclou el correu (Outlook), emmagatzematge (OneDrive) i aplicacions col·laboratives.
- **Salesforce:** La plataforma de gestió de clients (CRM) pionera en el món SaaS, utilitzada per milers d'empreses per gestionar les seves vendes completament des del navegador web.
- **Altres exemples universals:** *Google Workspace* (Gmail, Drive), *Zoom*, *Slack*, *Netflix* (a nivell de consum), o eines de disseny com *Figma*.

3.5. Resum general: La piràmide de serveis Cloud (IaaS vs. PaaS vs. SaaS)

La transició entre els tres models es pot resumir en la següent taula d'assignació de responsabilitats directes:

Característica / Recurs	On-Premise	IaaS	PaaS	SaaS
Aplicacions i Dades	Gestionat per tu	Gestionat per tu	Gestionat per tu	Gestionat pel proveïdor
Entorns d'execució (Runtime)	Gestionat per tu	Gestionat per tu	Gestionat pel proveïdor	Gestionat pel proveïdor
Sistema Operatiu (SO)	Gestionat per tu	Gestionat per tu	Gestionat pel proveïdor	Gestionat pel proveïdor
Maquinari, Disc i Xarxa	Gestionat per tu	Gestionat pel proveïdor	Gestionat pel proveïdor	Gestionat pel proveïdor

4. Arquitectura sense servidor (*Serverless*)

L'Arquitectura sense servidor (*Serverless*) és un dels models més avançats de computació en el núvol. Tot i el seu nom, sí que hi ha servidors; la diferència radical és que la gestió, l'aprovisionament i el manteniment d'aquests servidors queden completament ocults per a l'usuari. L'empresa es deslliga de la noció de "tenir una màquina virtual" i passa a enfocar-se en l'execució de fragments de codi integrats en una arquitectura elàstica.

4.1. Concepte de FaaS (Function-as-a-Service) i Computació basada en esdeveniments

El nucli tecnològic de Serverless és la **FaaS (Funció com a Servei)**. En lloc de desplegar una aplicació web sencera que està esperant de forma contínua que algú hi entri, el codi es divideix en funcions atòmiques i independents que només s'activen quan passa alguna cosa concreta. Aquest principi s'anomena **Computació basada en esdeveniments** (*Event-Driven Computing*).

- **Què és un esdeveniment?** Qualsevol acció digital dins del sistema que serveix com a disparador (*trigger*) per a executar la funció.
- **Exemples clau de disparadors:**
 - Un usuari prem un botó en una app mòbil que fa una petició HTTP.
 - S'acaba de pujar una imatge en un repositori d'emmagatzematge d'objectes (com AWS S3).
 - Arriba un missatge nou a una cua de processament de dades.

4.2. Funcionament operatiu del cicle de vida d'una funció

Quan es programa en un entorn Serverless, el cicle d'execució segueix de forma estricta aquests passos orquestrats pel proveïdor:

1. **Repòs:** El codi de la funció està guardat de forma passiva i **no consumeix recursos ni costa diners**.
2. **Disparador:** Ocorre l'esdeveniment (per exemple, un usuari puja una foto de perfil en format .raw).
3. **Arranque en fred (Cold Start):** El proveïdor de núvol detecta l'esdeveniment, crea instantàniament un contenidor micro-virtual invisible per sota, hi injecta el codi de la funció i l'executa.
4. **Execució i destrucció:** La funció fa la seva feina (per exemple, agafar la foto, reduir-li la mida a .jpg per a l'app i guardar-la a la base de dades). Un cop acaba la tasca (que sol durar uns pocs mil·lisegons o segons), el contenidor es destrueix immediatament.

4.3. Escalabilitat automàtica a zero (Scale-to-Zero)

L'atractiu principal de Serverless, tant a nivell tècnic com de model de negoci, és la seva capacitat d'escalat dinàmic i precís:

- **Escalabilitat instantània per ràfegues:** Si un sol usuari puja una imatge, s'executa una instància de la funció. Si de cop entren 10.000 usuaris en el mateix segon i pugen 10.000 imatges, el proveïdor de núvol aixeca 10.000 micro-contenidors en paral·lel per processar-les totes alhora. L'arquitecte no ha de configurar balancejadors de càrrega ni regles d'autoescalat complexos; el sistema ho fa sol.
- **Escalabilitat a zero:** Si durant la nit ningú utilitza l'aplicació, el nombre de funcions actives és exactament **zero**. Això significa que el cost de computació de l'empresa durant aquestes hores és de **zero euros**. És la màxima expressió de l'optimització de costos (*OpEx*), ja que es paga pel temps real de computació utilitzat calculat en fraccions de mil·lisegons.

4.4. Exemples de referència al mercat

- **Amazon Web Services (AWS): AWS Lambda** (el servei que va popularitzar el concepte Serverless al mercat global).
- **Google Cloud Platform (GCP): Google Cloud Functions / Cloud Run.**

- Microsoft Azure: Azure Functions.

4.5. Avantatges i Inconvenients del model Serverless

Beneficis clau (Avantatges)	Reptes tècnics (Inconvenients)
Zero administració: Oblida't d'actualitzar sistemes operatius, xarxes o servidors.	Latència del <i>Cold Start</i>: El primer cop que s'executa la funció després d'estar parada pot trigar unes dècimes de segon a arrencar.
Eficiència de costos: Només pagues quan el codi s'està executant (mesurat en mil·lisegons).	Límit de temps d'execució: Les funcions estan dissenyades per a tasques curtes (per exemple, AWS Lambda té un límit màxim de 15 minuts d'execució per petició).
Escalabilitat massiva: Respon a allaus de trànsit massiu sense intervenció humana.	Dificultat de depuració (<i>Debugging</i>): Trobar errors o provar l'aplicació sencera en local és complex a causa de la fragmentació del codi.

5. Matriu de responsabilitat compartida

El **Model de Responsabilitat Compartida** és el marc conceptual i legal que defineix quines tasques de seguretat, configuració i manteniment pertocuen al proveïdor de núvol (AWS, Azure, GCP) i quines recauen sobre el client (l'equip de **DevOps** o enginyeria de l'empresa).

A mesura que pugem en el nivell d'abstracció (de la IaaS a Serverless/SaaS), el proveïdor assumeix més control del sistema, cosa que allibera càrrega de feina operativa a l'equip de DevOps, però redueix la seva capacitat de personalització.

5.1. Matriu de Responsabilitat Compartida (Taula Comparativa)

La següent matriu detalla qui és el responsable de cada capa de la infraestructura segons el model de servei escollit:

Capa Tecnològica	On-Premise (Local)	IaaS (Infraestructura)	PaaS (Plataforma)	Serverless (FaaS)	SaaS (Programari)
Seguretat Física i Centres de Dades	Client	Proveïdor	Proveïdor	Proveïdor	Proveïdor
Maquinari, Servidors i Xarxa física	Client	Proveïdor	Proveïdor	Proveïdor	Proveïdor
Capa de Virtualització (Hypervisor)	Client	Proveïdor	Proveïdor	Proveïdor	Proveïdor
Sistema Operatiu (Pedaços i SO)	Client	Client (DevOps)	Proveïdor	Proveïdor	Proveïdor
Entorn d'execució (Runtime / Middleware)	Client	Client (DevOps)	Proveïdor	Proveïdor	Proveïdor
Lògica d'Escalabilitat i Redundància	Client	Client (DevOps)	Compartit	Proveïdor	Proveïdor
Codi de l'Aplicació i APIs	Client	Client (DevOps)	Client (DevOps)	Client (DevOps)	Proveïdor
Configuració	Client	Client (DevOps)	Client	Client	Client

d'Accessos i Identitats (IAM)			(DevOps)	(DevOps)	(DevOps)
Governança i Seguretat de les Dades	Client	Client (DevOps)	Client (DevOps)	Client (DevOps)	Client (DevOps)

5.2. Abast i límits de gestió de l'equip DevOps per model

A. Model IaaS (Ex. AWS EC2, Azure VMs)

- **Abast de DevOps: Màxim.** L'equip té control total per SSH/RDP. S'encarrega d'escollir la distribució de Linux/Windows, automatitzar les actualitzacions del sistema operatiu, configurar la seguretat de la xarxa virtual (tallafocs, VPC, subxarxes) i dissenyar des de zero l'arquitectura d'alta disponibilitat (instal·lant balancejadors i agents d'autoescalat).
- **Límits de DevOps:** No pot triar el maquinari físic de la marca del servidor ni intervenir si hi ha una fallada elèctrica o de disc físic al centre de dades del proveïdor.

B. Model PaaS (Ex. Heroku, AWS Elastic Beanstalk)

- **Abast de DevOps: Mig-Alt (Centrat en el desplegament).** El focus es desplaça cap a les canonades de CI/CD (integració i desplegament continu). DevOps configura variables d'entorn, gestiona l'accés d'usuaris a l'aplicació i defineix les regles de negoci per a l'autoescalat (ex: *"si la CPU de la plataforma passa del 70%, afegeix una altra instància"*).
- **Límits de DevOps:** No té accés a la línia de comandes del sistema operatiu base. No pot aplicar pedaços al *kernel* ni instal·lar programari que requereixi dependències profundes fora del *runtime* permès pel proveïdor.

C. Model Serverless / FaaS (Ex. AWS Lambda, Azure Functions)

- **Abast de DevOps: Focalitzat en l'arquitectura d'esdeveniments.** La feina de DevOps es converteix en "arquitectura de codi". S'encarrega de configurar correctament les connexions i els disparadors (*triggers*), gestionar les claus de seguretat de l'aplicació, monitoritzar el temps d'execució i controlar els costos de les mil·lisegons de còmput.
- **Límits de DevOps:** L'escalabilitat i la disponibilitat són 100% opaques; el proveïdor ho gestiona tot. DevOps no pot decidir en quin servidor físic ni en quin contenidor s'executa la funció, ni pot evitar el retard de l'arrencada en fred (*cold start*).

D. Model SaaS (Ex. Salesforce, Microsoft 365)

- **Abast de DevOps: Mínim (Rol d'Administrador d'Aplicació).** En aquest model, les tasques tradicionals de DevOps gairebé desapareixen. La gestió es limita a integrar el SaaS amb altres sistemes de l'empresa mitjançant APIs, gestionar el cicle de vida dels usuaris (altes/baixes) i assegurar que les dades que s'introdueixen a l'aplicació compleixen les polítiques de privacitat de la companyia.

- **Límits de DevOps:** L'equip no té accés al codi font de l'aplicació, no pot decidir quan es fa una actualització de programari ni pot modificar la base de dades interna del servei.

5.3. La regla d'or de la seguretat al Núvol

Per resumir l'abast d'aquesta matriu, la indústria utilitza una màxima senzilla per delimitar les fronteres:

El proveïdor és el responsable de la seguretat DEL núvol (la infraestructura, el ferro, els edificis i la virtualització).

El client (DevOps) és el responsable de la seguretat EN EL núvol (les seves dades, el seu codi, els seus sistemes operatius instal·lats i qui té permís per entrar-hi).

--ooOOoo--

Tema 5: Gestió de repositoris de codi, qualitat i seguretat en DevOps

-



1. Ecosistema d'eines per a contenidors
2. Eines d'anàlisi de qualitat i seguretat
3. Control d'accessos i seguretat en entorns (IAM)
4. Seguretat en la gestió del codi font
5. Automatització de proves de qualitat
6. Auditoria i escaneig de vulnerabilitats
7. Gestió i anàlisi de dependències

1. Ecosistema d'eines per a contenidors

L'ecosistema de contenidors ha transformat la manera d'empaquetar, distribuir i executar aplicacions. En lloc de dependre de configuracions manuals en servidors, aquest ecosistema utilitza un conjunt d'eines estandarditzades que assegurin que el programari funcione exactament igual en qualsevol entorn.

1.1. Motors de contenidors: L'entorn d'execució

El motor de contenidors (*container runtime*) és el programari encarregat d'executar i gestionar els contenidors de forma aïllada sobre el sistema operatiu amfitrió.

- **Docker:** L'estàndard històric que va popularitzar la tecnologia. És un ecosistema complet que inclou l'empaquetament, la gestió de xarxes, els volums i l'execució gràcies al seu component intern `dockerd`.
- **Containerd i CRI-O:** Motors d'execució d'alt rendiment, més lleugers i optimitzats. Són els que utilitza per baix **Kubernetes** en els entorns de producció moderns en lloc de Docker, ja que eliminen les capes de programari supèrflues per centrar-se exclusivament en l'execució eficient.
- **Podman:** Una alternativa de RedHat dissenyada per a ser un substitut directe de Docker, però amb un enfocament més segur: funciona **sense un dimoni central** (*daemonless*) i permet executar contenidors sense permisos de superusuari (*rootless*).

1.2. Empaquetament i format: Imatges de contenidors

L'empaquetament consisteix a consolidar l'aplicació, les seues dependències, llibreries i configuracions en un únic fitxer portàtil anomenat **Imatge**.

- **Estandardització (OCI):** Totes les imatges modernes segueixen la directiva de la *Open Container Initiative* (OCI). Això garanteix que una imatge creada amb Docker es pugui executar sense canvis en Podman o Kubernetes.
- **El Dockerfile:** És el plànol de construcció. Un fitxer de text on l'equip de DevOps defineix pas a pas com construir la imatge (trià la imatge base, copiar el codi de l'aplicació, instal·lar dependències i obrir ports).

1.3. Registres de contenidors: Emmatzematge i distribució

Un **Registre** és un magatzem centralitzat al núvol on es guarden, versionen i distribueixen les imatges de contenidors. Funciona com un "GitHub per a codi instal·lable".

- **Registres Públics:**
- **Docker Hub:** El registre oficial i el més gran del món. Conté milions d'imatges oficials (com les de Linux, Node.js, MySQL o Nginx) llistes per a ser utilitzades com a base per qualsevol desenvolupador.
- **Registres Privats i Corporatius (Ecosistemes CI/CD):**

- **GitHub Packages / GitLab Container Registry:** Integrats directament en les plataformes de codi. Permeten allotjar les imatges de l'empresa de forma privada al costat del codi font, facilitant l'automatització amb les teues canonades de desplegament.
- **Registres dels Hyperscalers:** Serveis nadius com *AWS ECR (Elastic Container Registry)* o *Azure ACR*. Ofereixen integració directa amb la seguretat del proveïdor del núvol, ideals per a desplegaments en producció dins del mateix núvol.

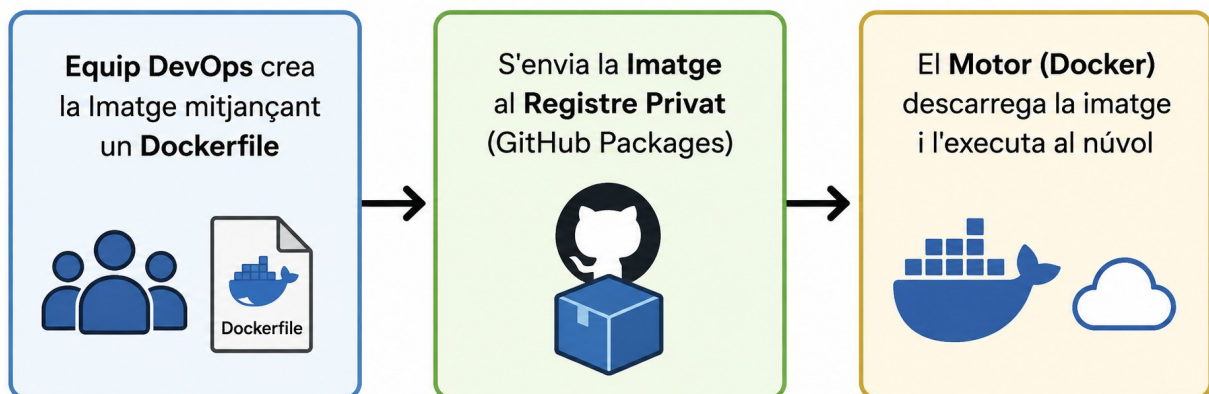
1.4. Gestió del desplegament i orquestració a petita escala

Abans de fer el salt a sistemes complexos com Kubernetes, l'ecosistema ofereix eines per a desplegar aplicacions en un sol servidor:

- **Docker Compose:** Una eina imprescindible per a DevOps que permet definir i executar **aplicacions multi-contenidor** utilitzant un únic fitxer de configuració en format YAML ([docker-compose.yml](#)).
- *Exemple d'ús:* En lloc d'aixecar de forma independent un servidor web, una base de dades i un sistema de memòria cau, Docker Compose els llança tots alhora interconnectats en una xarxa privada virtual en qüestió de segons amb una sola comanda ([docker compose up](#)).

1.5. Resum del cicle de vida en l'ecosistema de contenidors

El funcionament de l'ecosistema es pot resumir en un flux de tres passos clau (Build, Ship, Run):



2. Eines d'anàlisi de qualitat i seguretat

Dins de la metodologia DevOps, la seguretat i la qualitat del programari no es deixen per al final del projecte, sinó que s'integren de manera automatitzada des de l'inici. Aquest enfocament es coneix com a **Shift-Left** (moure la seguretat a l'esquerra del cicle de vida).

A continuació, es desenvolupa la classificació i el funcionament de les eines per a analitzar el codi sense necessitat d'executar-lo:

2.1. Eines SAST (Static Application Security Testing)

Les eines de Proves Estàtiques de Seguretat de l'Aplicació (SAST) analitzen el codi font, els binaris o els fitxers de configuració de l'aplicació en línia de repòs. Funcionen com un corrector ortogràfic avançat que busca vulnerabilitats de seguretat conegudes abans que el codi s'empaqueti o es desplegui.

- **Com funcionen?** Escanegen les línies de codi a la recerca de patrons perillosos definits en estàndards globals de seguretat (com el **Top 10 de l'OWASP**).
- **Què detecten?**
 - Injeccions de codi (SQL Injection, Cross-Site Scripting - XSS).
 - Gestió insegura de la memòria o desbordaments de búfer.
 - Ús d'algorismes de xifratge obsolets o febles.
 - Credencials inserides directament en el codi font (*hardcoded secrets*), com contrasenyes o claus d'APIs escrites per error.
- **Exemples clau del mercat:** *Snyk Code*, *Checkmarx*, *Veracode*, o alternatives de codi obert integrades com *Semgrep* i *Trivy* (per a fitxers d'infraestructura).

2.2. Eines de control de qualitat i deute tècnic

Més enllà de les esclatxes de seguretat, és vital mesurar la **salut i la sostenibilitat del codi**. Un codi desordenat, poc eficient o difícil d'entendre genera **deute tècnic** (costos futurs de manteniment).

Les eines de control de qualitat es fixen en:

- **Complexitat ciclomàtica:** Mesura quants camins possibles pot seguir l'execució d'una funció (massa bucles *if/else* niats fan que el codi sigui propens a errors).
- **Duplicació de codi:** Detecta blocs de text exactament iguals copiats en diferents fitxers, la qual cosa incompleix el principi *DRY (Don't Repeat Yourself)*.
- **Cobertura del test (Code Coverage):** Indica quin percentatge de les línies de codi de l'aplicació està realment cobert i comprovat per les proves unitàries automatitzades.
- **Mantenibilitat (Code Smells):** Identifica males pràctiques de programació que, tot i no ser errors de seguretat directes, fan que el codi sigui fràgil i difícil de modificar.

2.3. SonarQube: El referent en la inspecció contínua

SonarQube és la plataforma central d'inspecció de qualitat de codi més utilitzada en entorns DevOps. Destaca per la seva capacitat d'aglutinar l'anàlisi estàtica de qualitat i seguretat en un únic panell centralitzat:

- **Les Portes de Qualitat (Quality Gates):** Permet definir regles estrictes a la canonada de CI/CD. Per exemple: *"Si el codi nou té una cobertura de tests inferior al 80% o conté algun*

error de seguretat de nivell crític, bloqueja automàticament el desplegament i no el deixis passar a producció".

- **Suport multi-llenguatge:** És capaç d'analitzar projectes que barregen desenes de llenguatges diferents (Java, Python, JavaScript, C#, etc.) de forma simultània.

3. Control d'accessos i seguretat en entorns (IAM)

El Control d'Accessos i la Gestió d'Identitats (IAM - Identity and Access Management) és la primera i més important línia de defensa en qualsevol infraestructura tecnològica. En la metodologia DevOps, on els desplegaments són ràpids i automatitzats, un disseny feble dels permisos pot exposar dades crítiques de l'empresa o permetre canvis no autoritzats que tombin els serveis en producció.

A continuació, es desenvolupen els pilars clau de la governança i seguretat d'accessos:

3.1. Conceptes fonamentals d'IAM: Usuaris, Grups i Rols

La gestió d'accessos en el núvol i en plataformes de codi s'organitza mitjançant estructures lògiques ben definides:

- **Usuaris (Users):** Representen la identitat d'una persona real (un desenvolupador, un administrador de bases de dades). Cada usuari ha de tenir les seves pròpies credencials i mètodes d'autenticació multifactor (**MFA**) activats obligatòriament.
- **Grups (Groups):** Conjunts d'usuaris que comparteixen una mateixa funció (ex. "Equip de Disseny Web", "Equip de Ciberseguretat"). És una bona pràctica assignar permisos als grups i no als usuaris individuals per simplificar la gestió.
- **Rols (Roles):** Identitats abstractes que no pertanyen a cap persona en concret. Els rols s'utilitzen perquè **les màquines, serveis o aplicacions demanin permís temporal** per fer una feina.
- *Exemple:* Un servidor web (IaaS) pot rebre temporalment el "Rol de Lector de Fitxers" per poder accedir a l'emmagatzematge d'objectes (SaaS) sense necessitat de guardar cap contrasenya escrita al seu codi.

3.2. El Principi del Mínim Privilegi (Least Privilege Principle)

Aquest és el pilar de seguretat més crític de la matriu IAM. Estableix que **tot usuari, servei o procés ha de tenir exclusivament els permisos estrictes que necessita per fer la seva feina, i cap més.**

- **Evitar els superusuaris (Admin/Root):** Cap desenvolupador ni administrador hauria d'utilitzar el compte *root* per a tasques diàries.
- **Permissos granulars:** En lloc d'atorgar accés a "Tota la base de dades", s'ha de configurar l'IAM per permetre només "Accés de lectura a la taula de catàleg de productes". Si el compte d'un desenvolupador es veu compromès per un atac de *phishing*, l'impacte en l'empresa es veurà limitat al mínim gràcies a aquest aïllament.

3.3. Assignació d'accessos i segregació d'entorns de producció

La infraestructura s'ha de dividir conceptualment i tècnicament per protegir el cor de l'empresa: els entorns de producció (on hi ha els clients reals).

- **Segregació estricta de comptes:** L'entorn de Desenvolupament (Dev), Proves (Staging) i Producció (Prod) han d'estar en **xarxes (VPCs) o comptes de núvol completament separats**. Les claus d'accés a desenvolupament mai han de servir per a producció.
- **Accés automatitzat per davant de l'humà (CI/CD):** En entorns moderns de DevOps, els humans **no han de tenir permisos d'escriptura a producció**. Qui fa els canvis, aixeca servidors o esborra dades és la canonada automatitzada de desplegament continu (ex. *GitHub Actions*, *GitLab CI*). L'equip de DevOps només aprova el codi en el repositori central.
- **Accessos d'emergència (*Break-Glass / Bastion*):** Si hi ha un incident greu i un enginyer necessita entrar directament a producció a resoldre'l, s'utilitzen procediments especials d'auditoria: l'accés s'ha de sol·licitar formalment, s'atorga de forma **temporal** (per exemple, durant dues hores) i totes les accions que realitzi l'humà queden registrades en vídeo o en *logs* inalterables per a una posterior revisió.

3.4. Resum Tècnic: Matriu d'Accés per Rol de Negoci

Rol del Professional	Permís en Entorn de Desenvolupament	Permís en Entorn de Producció
Desenvolupador (Developer)	Full Control (Crear/Eliminar recursos de prova).	Read-Only (Només veure gràfics de rendiment o mètriques).
Línia CI/CD (Robot / Runner)	Read/Write (Automatitzat per a provar el codi).	Read/Write (Únic encarregat de desplegar canvis aprovats).
Enginyer DevOps / Administrador	Full Control (Manteniment de les estructures).	Read-Only (<i>Break-Glass</i> per a emergències).

4. Seguretat en la gestió del codi font

La gestió del codi font és el punt d'origen de tota la cadena de subministrament de programari (*software supply chain*). Protegir el repositori Git no només evita la pèrdua de codi, sinó que impedeix que atacants externs injectin programari maliciós en el sistema o furten claus d'accés crítiques de l'empresa.

A continuació, es desenvolupen les estratègies i configuracions clau per a un entorn Git segur:

4.1. Definició de branques protegides (*Branch Protection Rules*)

Les branques principals (com `main` o `production`) representen el codi que s'executa o s'executarà directament als servidors reals. Per tant, cap humà ni robot hauria de poder modificar-les directament sense supervisió.

Les regles de protecció s'apliquen per garantir que tot canvi sigui auditat:

- **Requisit de Pull Request (PR):** Es prohibeixen els commits directes (`git push`) a la branca protegida. Qualsevol canvi s'ha de fer en una branca secundària i sol·licitar la integració mitjançant una PR.

- **Revisió per parells (Code Review):** Es requereix l'aprovació d'un o diversos desenvolupadors sèniors abans de fusionar el codi. Això ajuda a detectar errors de lògica o codi maliciós passat per alt.
- **Comprovacions d'estat obligatòries (Status Checks):** La branca protegida no acceptarà el nou codi si la línia de CI/CD (ex. *GitHub Actions*) falla. S'ha de garantir que els tests unitaris, les anàlisis SAST (ex. *SonarQube*) i els d'estil s'executin correctament abans de la fusió.
- **Bloqueig de reescriptures d'historial:** Es prohibeixen els canvis destructius com el `git push --force`, que podrien esborrar l'historial de revisions d'un projecte.

4.2. Polítiques d'accés i auditoria al repositori

La governança de qui pot veure o modificar el codi s'organitza sota un control estricte del proveïdor de Git (*GitHub*, *GitLab* o *Bitbucket*):

- **Autenticació mitjançant Claus SSH i Signatura de Commits:** Els desenvolupadors de l'empresa s'han d'autenticar utilitzant claus SSH fortes o tokens personals de curta durada (PAT). A més, es recomana activar la **signatura GPG de commits**, que col·loca una etiqueta de verificació a Git per garantir que l'autor del codi és realment qui diu ser, evitant la suplantació d'identitat.
- **Equips i Rols basats en SAML/SSO:** Els permisos dels repositoris es vinculen a l'entitat d'identitat de l'empresa (ex. *Entra ID*, *Okta*). Si un programador canvia de departament o plega, el seu accés al codi es revoca instantàniament des del panell central.
- **Registres d'auditoria (Audit Logs):** El sistema registra cada descàrrega de codi (`git clone`), cada accés de tercers i qualsevol canvi en la configuració del repositori per a futures investigacions de ciberseguretat.

4.3. Gestió de secrets i paràmetres segurs

Un dels errors més comuns i catastròfics en DevOps és el **"hardcoding"**: escriure contrasenyes, claus d'APIs o credencials de bases de dades directament dins de fitxers de codi (com un fitxer `.js` o un `Dockerfile`) i pujar-los a Git. Si el repositori es fa públic o es veu compromès, l'empresa queda totalment exposada.

Per evitar-ho, s'apliquen tres capes de seguretat tecnològica:

- **Separació de configuració i codi:** Totes les credencials s'extreuen del codi i es llegeixen en temps d'execució mitjançant **variables d'entorn** o paràmetres injectats de forma dinàmica.
- **Secrets de la línia CI/CD (GitHub Secrets / GitLab CI Variables):** Les credencials que necessita el robot de desplegament per connectar-se al núvol es guarden xifrades a la configuració del repositori. El sistema les oculta dels *logs* i de la vista dels desenvolupadors comuns, utilitzant-les només quan s'executa la canonada d'automatització.
- **Eines natis de detecció de fuites (Secret Scanning):** S'activen eines automatitzades com *GitGuardian*, *GitHub Secret Scanning* o *TruffleHog*. Aquests sistemes analitzen cada `git`

push a temps real. Si detecten per patró que un programador ha inclòs una clau privada per descuit, bloquegen el commit de forma immediata o revoquen la clau per evitar que sigui utilitzada.

4.4. Resum de bones pràctiques en Repositoris

Risc de Seguretat	Mesura Tècnica de Prevenció
Un desenvolupador puja codi trencat o vulnerable directament a producció.	Branques protegides amb revisió per parells i <i>Status Checks</i> de CI/CD obligatoris.
Un atacant es fa passar per un empleat i puja un virus al codi.	Exigir autenticació MFA i activació de la signatura GPG de commits .
S'ha pujat accidentalment la clau d'AWS al repositori de l'empresa.	Secret Scanning actiu i ús de variables d'entorn xifrades (GitHub Secrets).

5. Automatització de proves de qualitat

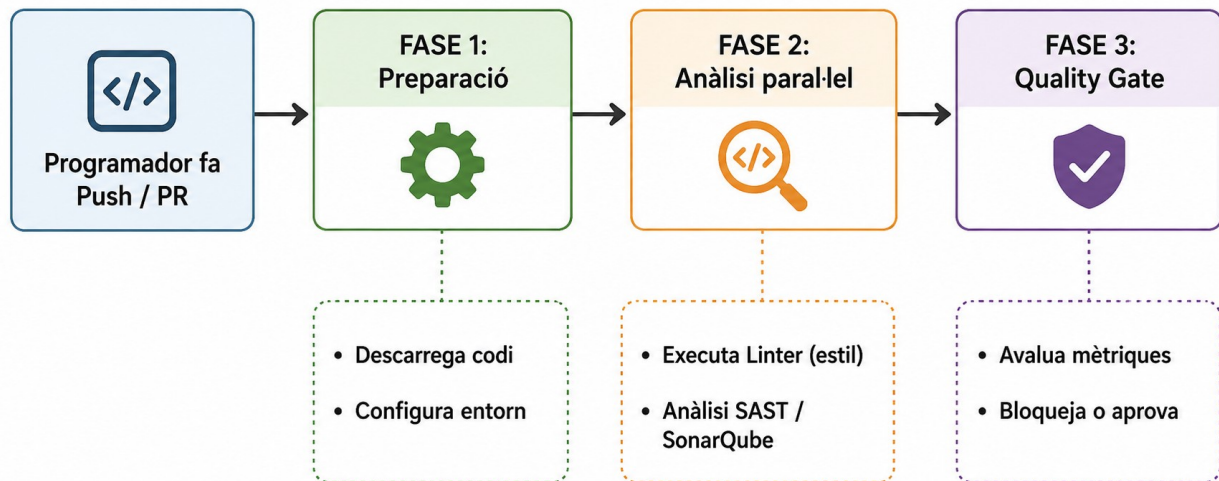
L'automatització de proves de qualitat és la materialització pràctica de la filosofia *Shift-Left*. En lloc d'executar l'anàlisi de qualitat i seguretat de forma manual o periòdica (un cop al mes), aquestes proves s'integren de manera nativa com a requisits binaris i automatitzats dins de la canonada de desplegament (*CI/CD pipeline*). Si el codi d'un desenvolupador no supera els estàndards, el sistema frena el procés abans que pugui arribar a producció.

A continuació, es desenvolupa la integració tècnica d'aquests fluxos i la generació d'informes:

5.1. Integració de proves estàtiques dins del flux de treball (CI/CD)

L'anàlisi estàtica de codi (que inclou linters d'estil, SAST i mètriques de qualitat) s'executa automàticament en moments clau del cicle de vida del desenvolupament gràcies als *runners* o robots d'automatització (com *GitHub Actions*, *GitLab CI*, o *Jenkins*).

El flux tècnic estàndard es divideix en tres fases d'execució:



1. **Fase de Preparació:** El robot de CI/CD detecta que s'ha obert una *Pull Request*. Descarrega el codi nou en un contenidor efímer aïllat i configura l'entorn bàsic de l'aplicació.
2. **Fase d'Anàlisi en Paral·lel:** El sistema llança les eines d'inspecció de forma simultània per estalviar temps:
 - **Linters:** Comproven que el codi segueix les regles d'estil de la companyia (estructures de fitxers, noms de variables).
 - **Escàner SAST/Qualitat (ex. SonarQube Scanner):** Analitza el codi a la recerca de deute tècnic, errors lògics o vulnerabilitats OWASP.
3. **Fase de decisió (The Quality Gate):** L'orquestrador de CI/CD avalua els resultats de l'anàlisi en funció de les regles de negoci de l'empresa.

5.2. El concepte de Quality Gate (La tanca de control)

El *Quality Gate* és un conjunt de llindars que el codi nou ha de complir obligatòriament per ser considerat vàlid. Si una sola de les mètriques cau per sota del límit, la canonada de CI/CD es dona per fallida, es notifica al desenvolupador i es bloqueja la fusió (*merge*) del codi.

Els llindars típics configurats per un equip DevOps són:

- **Errors de seguretat (Vulnerabilitats):** Zero errors crítics o alts permesos.
- **Cobertura de proves (Code Coverage):** El codi nou ha d'estar cobert per tests unitaris almenys en un **80%**.
- **Codi duplicat:** Menys d'un **3%** de línies duplicades a tot el projecte.
- **Deute tècnic:** Qualificació (Rating) de Mantenibilitat de nivell "A".

5.3. Generació d'informes de validació previs al desplegament

L'automatització de les proves no només serveix per bloquejar codi dolent, sinó per generar **evidències i artefactes d'auditoria** que certifiquen la salut de l'aplicació abans de prémer el botó de desplegar.

- **Informes executius en taulers (Dashboards):** Eines com SonarQube o Snyk generen gràfics visuals immediats un cop acaba la prova. Això permet als líders tècnics avaluar la salut global del projecte al llarg del temps, veure quines branques són més estables i fer un seguiment de com evoluciona el deute tècnic.
- **Ressenyes automatitzades a la PR (Pull Request Comments):** El robot d'anàlisi utilitza APIs per escriure directament dins de la interfície de GitHub o GitLab. Si detecta una línia de codi vulnerable, col·loca un comentari automàtic indicant al programador: *"A la línia 42 estàs fent una injecció SQL. Soluciona-ho fent servir consultes preparades"*.
- **Generació de fitxers d'auditoria (SARIF / JSON):** Les proves generen informes en formats estàndard del sector com **SARIF** (*Static Analysis Results Interchange Format*). Aquests fitxers es guarden com a artefactes de la línia de CI/CD i serveixen perquè els equips de ciberseguretat de l'empresa puguin auditar de forma externa què s'ha enviat exactament a producció i amb quines garanties es va fer.

6. Auditoria i escaneig de vulnerabilitats

Dins del cicle de vida del desenvolupament segur, no n'hi ha prou amb analitzar el codi en el moment exacte en què es programa o es desplega. L'**auditoria i l'escaneig periòdic de vulnerabilitats** és la pràctica que garanteix la seguretat a llarg termini. El motiu és senzill: un codi que avui es considera 100% segur i lliure d'errors pot esdevenir vulnerable demà si es descobreix una nova esquerda de seguretat (una vulnerabilitat *Zero-Day*) en una de les llibreries que utilitza.

6.1. El concepte d'escaneig periòdic asíncron

Mentre que les proves en les línies de CI/CD són reactives (s'activen només quan un programador fa canvis), l'auditoria periòdica és **proactiva i automatitzada**.

- **Execució programada (Cron Jobs):** Les eines d'escaneig es configuren perquè s'executin de forma automàtica en segon pla en intervals fixos (per exemple, cada nit a les 02:00 AM o un cop per setmana).
- **Anàlisi del codi en repòs:** L'escàner analitza els repositoris principals (main o production) de forma independent a l'activitat de l'equip de desenvolupament, creuant la informació del codi amb bases de dades de ciberseguretat globals actualitzades diàriament (com la base de dades **CVE** - *Common Vulnerabilities and Exposures*).

6.2. Eines d'anàlisi de composició de programari (SCA)

Gran part de l'auditoria periòdica es basa en les eines **SCA (Software Composition Analysis)**. En una aplicació moderna, prop del 80% del codi final prové de llibreries de tercers i paquets de codi obert (descarregats de repositoris com *npm*, *pip* o *Maven*). L'eina SCA s'encarrega d'auditar aquesta cadena de subministrament:

- **Dependabot:** Eina nativa de GitHub que escaneja constantment els fitxers de dependències (com `package.json` o `requirements.txt`). Si es descobreix una vulnerabilitat en una llibreria, Dependabot obre automàticament una *Pull Request* que actualitza la llibreria a una versió segura, reduint la feina de l'equip de DevOps al mínim.
- **Snyk / Sonatype Nexus:** Eines empresarials avançades que no només busquen vulnerabilitats en les llibreries de codi, sinó que també auditen les **imatges de contenidors (Docker)** i el codi d'**Infraestructura com a Codi (Terraform)** a la recerca de males configuracions de seguretat.
- **Auditoria de llicències legals:** Aquestes eines comproven que les llibreries utilitzades no incompleixin les polítiques de llicències de l'empresa (per exemple, alertar si s'utilitza una llibreria amb llicència GPL que obligaria a fer públic el codi privat de la companyia).

6.3. Triage i gestió de vulnerabilitats

Quan l'escaneig periòdic detecta una vulnerabilitat, s'activa un flux de governança per evitar el col·lapse de l'equip de seguretat. Les vulnerabilitats es classifiquen segons el sistema d'escala estàndard **CVSS** (*Common Vulnerability Scoring System*):

1. **Crítica / Alta (Critical / High):** Escletxes que permeten l'execució de codi remot o el robatori massiu de dades sense necessitat d'autenticació. Generen una **alerta immediata** als equips de guàrdia (mitjançant *Slack*, *PagerDuty* o *Jira*) i requereixen un pedaç en qüestió d'hores.
2. **Mitjana / Baixa (Medium / Low):** Problemes menors que requereixen condicions molt específiques per ser explotats. Es registren en el tauler de deute tècnic i es planifica la seva resolució per al següent esprint de treball.
3. **Falsos Positius:** Situacions on l'eina detecta un patró perillós, però l'arquitectura de l'empresa impedeix que s'exploti. L'equip de DevOps ha de documentar i marcar el cas com a "ignorant" perquè no distregui en futurs escanejos.

7. Gestió i anàlisi de dependències

El darrer pilar de la seguretat en el desenvolupament de programari és el control de la cadena de subministrament. Com que les aplicacions modernes es construeixen connectant peces preexistents, la gestió i l'anàlisi de dependències mitjançant eines SCA (Software Composition Analysis) és l'única manera de garantir que el codi extern que s'introdueix al projecte és segur, legal i fiable.

A continuació, es desenvolupa com funcionen i s'apliquen les tecnologies SCA:

7.1. El problema del codi aliè: Dependències directes i transitives

Quan un programador afegeix una llibreria externa (per exemple, per a gestionar dates o validar formularis), està introduint dues capes de codi que cal auditar:

- **Dependències directes:** Les llibreries que l'equip escull i afegeix explícitament al fitxer de configuració del projecte (com package.json en Node.js, pom.xml en Java o requirements.txt en Python).
- **Dependències transitives (o en cascada):** Són les llibreries que la teva llibreria necessita per a funcionar. Un projecte que només afegeix 5 dependències directes pot acabar executant **centenars de llibreries transitives** de forma oculta, ampliant radicalment la superfície d'atac.

7.2. Com funciona una eina SCA?

El programari SCA (com **Snyk**, **Sonatype Nexus**, **WhiteSource/Mend** o **Dependabot**) s'integra en el repositori de codi i realitza una inspecció en tres nivells:

A. Verificació de vulnerabilitats (CVEs)

L'eina extreu l'inventari complet de totes les llibreries (directes i transitives) i les seves versions exactes (generant el que es coneix com a **SBOM - Software Bill of Materials** o llista de components de programari). Després, encreua aquesta llista amb bases de dades globals de vulnerabilitats conegudes (com la *National Vulnerability Database* o NVD). Si detecta un paquet compromès, emet una alerta indicant quin és el perill i quina versió el soluciona.

B. Auditoria i compliment de llicències legals

No tot el codi obert es pot utilitzar lliurement en entorns empresarials. Les eines SCA analitzen les llicències de cada llibreria per evitar problemes legals:

- **Llicències Permissives (ex. MIT, Apache 2.0):** Permeten l'ús comercial i privat del codi sense restriccions. L'eina les aprova automàticament.
- **Llicències Copyleft / Protectores (ex. GPLv3):** Obliguen l'empresa a fer públic el codi font de tot el seu producte si utilitza eixa llibreria. L'eina SCA pot bloquejar la línia de CI/CD si un programador intenta introduir una llicència restrictiva per error.

C. Control d'obsolescència i salut del paquet

Més enllà dels errors o de la llei, l'SCA avalua si una llibreria externa és de confiança analitzant el seu manteniment: detecta si el paquet està abandonat pel seu creador, si fa anys que no rep actualitzacions o si té una comunitat activa, evitant que l'empresa hereti codi mort difícil de mantenir.

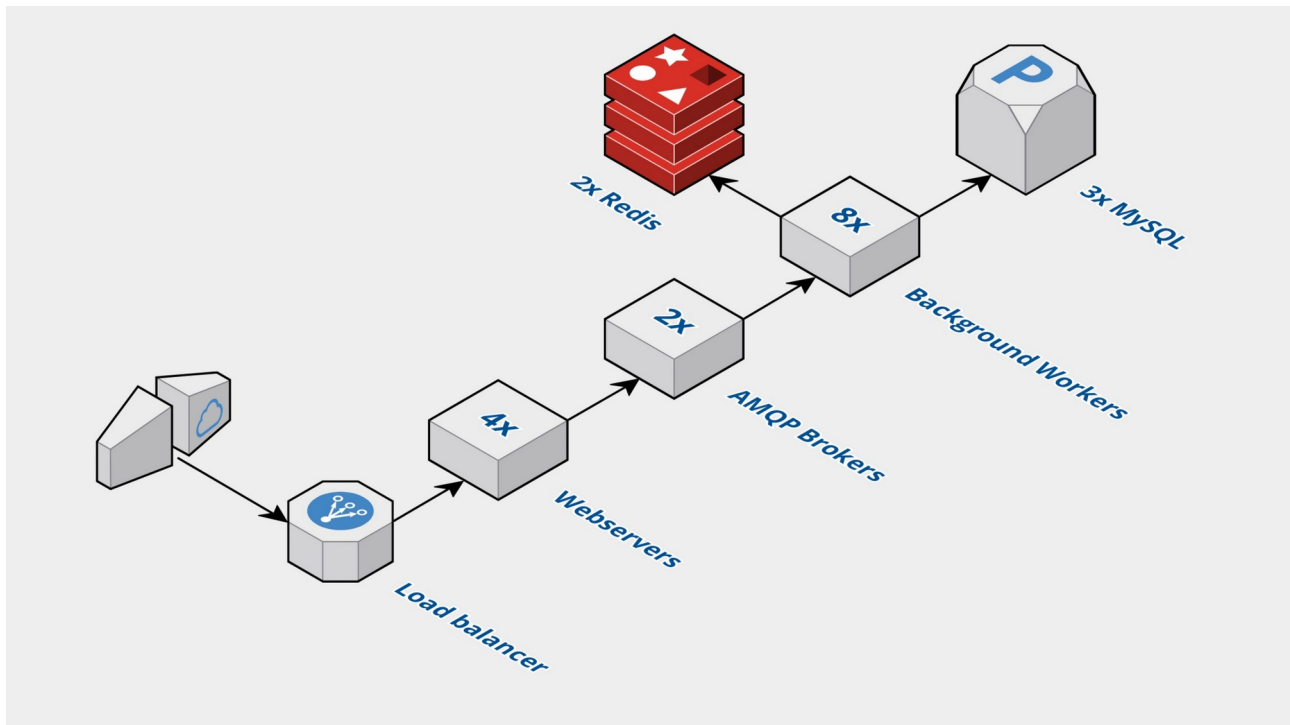
7.3. Integració i automatització del flux de treball

L'anàlisi de dependències funciona de manera dual en un entorn DevOps eficient:

1. **Bloqueig en la línia de CI/CD (Reactiva):** Quan es crea una *Pull Request*, l'SCA escaneja el codi nou. Si es detecta una nova dependència amb una vulnerabilitat crítica, el *Quality Gate* es trenca i el codi no es pot fusionar.
2. **Pedaços automatitzats en repòs (Proactiva):** Eines com **Dependabot** cerquen actualitzacions de forma constant. Si surt un pedaç de seguretat, l'eina crea automàticament una branca, actualitza el fitxer de dependències, llança els tests de l'empresa per comprovar que res no s'ha trencat i deixa la *Pull Request* llesta perquè l'equip de DevOps només hagi de prémer el botó d'aprovar.

--ooOOoo--

Tema 6: Infraestructura com a Codi (IaC)



1. Introducció a l'Infraestructura com a Codi (IaC)
2. Creació de fitxers de configuració IaC
3. Creació de fluxos de treball (Pipelines) d'infraestructura
4. Polítiques de seguretat digital en entorns
5. Desplegament automatitzat d'infraestructura
6. Validació de l'entorn i inspecció postdesplegament

1. Introducció a l'Infraestructura com a Codi (IaC)

L'Infraestructura com a Codi (IaC) és la pràctica que defineix la gestió i l'aprovisionament dels recursos tecnològics (servidors, xarxes, emmagatzematge, bases de dades) mitjançant fitxers de configuració legibles per humans i màquines, en lloc de fer-ho mitjançant processos manuals o eines gràfiques interactives.

1.1. El concepte de definició de maquinari mitjançant programari (SDH)

Històricament, per muntar un servidor calia que un operador anés físicament al centre de dades, cargolés la màquina en un *rack*, connectés els cables de xarxa i instal·lés el sistema operatiu. Amb el naixement del núvol i de les xarxes definides per programari (SDN), el maquinari es converteix en un recurs abstracte i purament digital.

Aquest canvi es consolida amb el concepte de Maquinari Definit per Programari (SDH - *Software Defined Hardware*):

- **El maquinari és codi:** Com que la infraestructura del núvol és accessible mitjançant APIs, podem programar la creació d'un centre de dades sencer escrivint unes línies de text en un ordinador.
- **L'automatització com a norma:** La infraestructura es pot versionar, provar i replicar de forma idèntica exactament igual que es fa amb el codi de qualsevol aplicació de programari.

1.2. El canvi de paradigma en la gestió d'infraestructures

L'aplicació d'IaC ofereix tres grans propietats tècniques:

- **Idempotència:** És la capacitat d'executar el mateix fitxer de configuració de la IaC desenes de vegades garantint que **el resultat final serà sempre exactament el mateix**, sense crear recursos duplicats ni errors.
- **Consistència:** Elimina l'error humà. Ja no hi ha perill que un administrador s'oblidi de marcar una casella en un panell web a l'hora de configurar l'entorn de producció, fent que sigui diferent al de desenvolupament.
- **Documentació viva:** El propi fitxer de codi és el plànol actualitzat de la xarxa de l'empresa. Si algú vol saber com està configurat un tallafocs, només cal que miri el codi de la IaC.

1.3. Llenguatges Declaratius vs. Imperatius en IaC

A l'hora de dissenyar eines d'IaC, el mercat es divideix en dues grans filosofies de programació segons com s'indiquen les instruccions al sistema:

FILOSOFIES DE LENGUATGE	
MODEL IMPERATIU ("El Com")	MODEL DECLARATIU ("El Què")
"Crea la màquina A, espera 5 minuts, instal·la la xarxa B, connecta-les"	"Vull tenir 3 màquines del tipus X connectades a la xarxa Y. Tu decideix els passos."

A. Model Imperatiu (Enfocament basat en "El Com")

L'usuari especifica els **passos exactes i l'ordre correcte** que l'eina ha de seguir per arribar a l'estat desitjat. S'assembla a una recepta de cuina o a un script de comandes.

- *Com funciona:* Si vols aixecar 3 servidors, l'script conté un bucle que diu: "executa la comanda de creació de servidor, comprova si ha acabat, i repeteix-ho dues vegades més".
- *Inconvenient:* Si l'script falla a la meitat, l'entorn es queda en un estat trencat o desconegut, i tornar-lo a executar pot provocar errors o duplicar servidors.
- *Eines de referència:* AWS CLI, scripts de Bash, o algunes configuracions d'Ansible (tot i que Ansible és híbrid).

B. Model Declaratiu (Enfocament basat en "El Què")

L'usuari defineix l'**estat final desitjat de la infraestructura**, i és la pròpia eina la que s'encarrega d'analitzar la situació actual, calcular la diferència i realitzar de forma intel·ligent les accions necessàries per a aconseguir-ho.

- *Com funciona:* Escrius al document: "Vull que hi hagi 3 servidors de tipus EC2 i una base de dades". Si l'eina detecta que actualment només n'hi ha 2, no esborrarà res; simplement crearà el servidor que falta per complir la declaració.
- *Avantatge:* És el model estàndard en DevOps professionals gràcies a la seva alta seguretat, mantenibilitat i estabilitat.
- *Eines de referència:* **Terraform**, AWS CloudFormation, o els fitxers de configuració de Kubernetes (YAML).

2. Creació de fitxers de configuració IaC

La creació pràctica d'infraestructura com a codi es basa en l'escriptura de fitxers de text utilitzant sintaxis estructurades (com **HCL**, **YAML** o **JSON**). L'objectiu és separar la lògica de l'arquitectura dels valors concrets (com les mides de les màquines o les contrasenyes) mitjançant **fitxers de paràmetres o variables**. Això permet reutilitzar el mateix codi per a crear entorns idèntics de Desenvolupament o Producció de forma segura.

A continuació, es desenvolupa la sintaxi, el cicle de vida i l'ús pràctic de les dues eines líders del mercat:

2.1. Terraform: El referent multinúvol (Sintaxi HCL)

Terraform (de HashiCorp) és una eina declarativa que destaca per ser compatible amb gairebé qualsevol proveïdor (AWS, Azure, GCP, VMware) gràcies a l'ús de **Providers** (connectors oficials). Utilitza el llenguatge **HCL** (*HashiCorp Configuration Language*), dissenyat per a ser molt llegible.

El Cicle de Vida del recurs a Terraform

Per a crear qualsevol infraestructura, un enginyer DevOps executa una seqüència de quatre comandos clau:

1. terraform init: Descarrega els *providers* (ex: el connector d'AWS) i inicialitza l'entorn de treball.
2. terraform plan: L'eina llegeix el teu codi, el compara amb la realitat del núvol i mostra un **esborrany d'accions** (què crearà, què modificarà i què esborrarà) sense aplicar cap canvi encara.
3. terraform apply: Executa les accions aprovades en el *plan* i crea de veritat els recursos a l'API del proveïdor.
4. terraform destroy: Esborra tota la infraestructura definida en el fitxer de codi amb un sol clic.

Exemple pràctic de codi i paràmetres a Terraform

Per a evitar escriure valors fixos al codi principal, s'utilitza una estructura de tres fitxers separats:

- **Fitxer 1: Definició de Variables (variables.tf)** (Declara quins paràmetres existiran):

```
variable "tipus_servidor" {  
  type      = string  
  description = "Mida de la màquina virtual"  
}
```

- **Fitxer 2: Codi Principal d'Arquitectura (main.tf)** (Defineix l'estructura utilitzant la variable):

```
resource "aws_instance" "el_meu_servidor" {
  ami          = "ami-0c55b159cbfaffe1f0" # Imatge base d'Ubuntu
  instance_type = var.tipus_servidor      # Crida a la variable
}
```

- **Fitxer 3: Fitxer de Paràmetres (terraform.tfvars)** (On s'assigna el valor real segons l'entorn):

```
# Si estem a Desenvolupament, assignem una màquina xicoteta i barata:
tipus_servidor = "t2.micro"
```

2.2. AWS CloudFormation: La solució nativa d'Amazon (Sintaxi YAML)

CloudFormation és l'eina declarativa nativa d'Amazon Web Services. No requereix instal·lar cap programari extern, ja que s'executa directament dins de la consola d'AWS. El codi s'organitza en Stacks (piles de recursos) i s'escriu habitualment en YAML.

Exemple pràctic de codi i paràmetres a CloudFormation

Igual que Terraform, CloudFormation permet separar els paràmetres dins del mateix document o utilitzar un fitxer de configuració de paràmetres JSON independent:

- **Fitxer de plantilla principal (template.yaml):**

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Creació d'un servidor bàsic amb CloudFormation'

# 1. Declara el paràmetre
Parameters:
  TipusServidor:
    Type: String
    Default: t2.micro
    Description: Mida de la instància d'AWS

# 2. Defineix el recurs enllaçat al paràmetre
Resources:
  ElMeuServidorEC2:
    Type: 'AWS::EC2::Instance'
    Properties:
      ImageId: 'ami-0c55b159cbfaffe1f0'
      InstanceType: !Ref TipusServidor # Fa referència al paràmetre superior
```

- Fitxer de paràmetres per a l'automatització (dev-params.json):

```
[
  {
    "ParameterKey": "TipusServidor",
    "ParameterValue": "t2.micro"
  }
]
```

2.3. Comparativa de Control i Ús Pràctic

Criteri de Selecció	Terraform (Multinúvol)	AWS CloudFormation (Natiu)
Abast del proveïdor	Qualsevol núvol (AWS, Azure, GCP) i entorns locals.	Exclusiu d'AWS. No serveix per a cap altre núvol.
Gestió de l'Estat (State)	Guarda un fitxer local o remot (<u>.tfstate</u>) que recorda exactament com està el núvol. Cal protegir-lo.	Gestionat 100% de forma invisible i automàtica per AWS dins de l'Stack.
Llenguatge	HCL (Molt concís, menys línies de codi).	YAML / JSON (Més visual, però fàcil de fer-se molt llarg).
Lògica de variables	Utilitza fitxers <u>.tfvars</u> altament modulars.	Utilitza seccions <u>Parameters</u> o fitxers externs JSON.

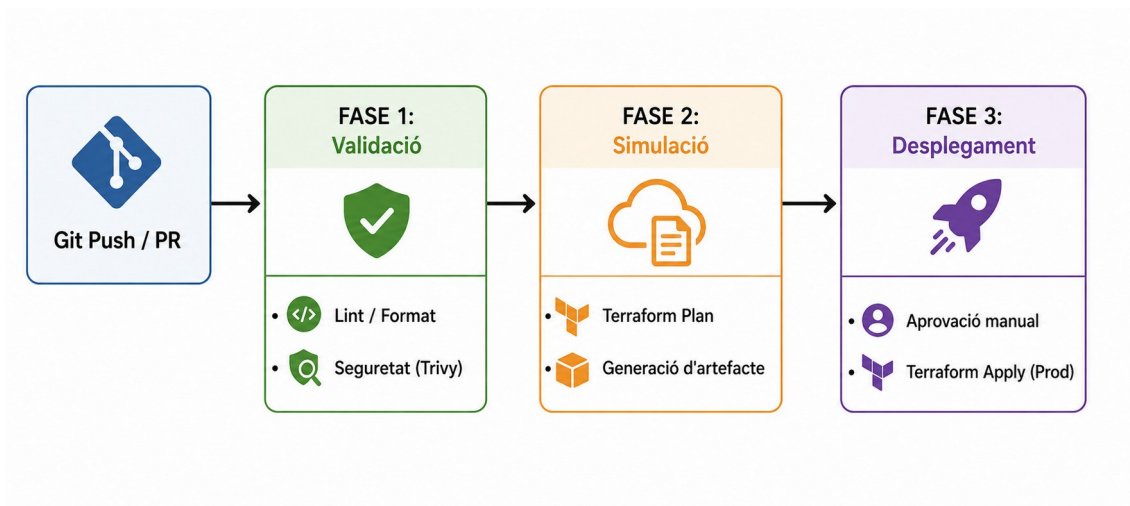
3. Creació de fluxos de treball (Pipelines) d'infraestructura

El disseny de fluxos de treball (Pipelines) d'infraestructura (sovint anomenat **GitOps** o *CI/CD per a IaC*) és el pas definitiu per a eliminar la interrupció manual en la gestió de sistemes. En lloc d'executar comandes com `terraform apply` des de l'ordinador d'un enginyer, és un robot d'automatització (com *GitHub Actions*, *GitLab CI* o *Jenkins*) qui s'encarrega de connectar-se al proveïdor de núvol i desplegar el maquinari de forma segura i auditable.

A continuació, es desenvolupa el disseny d'aquests fluxos i la gestió de les seues credencials:

3.1. El disseny de la Pipeline d'infraestructura

Una línia d'automatització d'IaC no es limita a executar el codi; ha de seguir una sèrie de passos de validació per a garantir que no s'introdueixen errors crítics a la xarxa:



1. Fase de Validació i Seguretat:

- **Linting:** Es comprova que la sintaxi del codi és correcta (ex. `terraform fmt -check`).
- **Anàlisi de seguretat de la IaC:** S'utilitzen escàners estàtics (com *Trivy*, *Checkov* o *KICS*) per a analitzar el fitxer de configuració a la recerca de males pràctiques abans de crear res (per exemple, alertar si un fitxer de Terraform està intentant obrir el port de bases de dades 3306 a tot Internet).

2. Fase de Simulació (*Plan*):

- El robot executa la comanda de simulació (ex. `terraform plan`). El resultat es publica com un comentari automàtic dins de la *Pull Request* perquè l'equip de DevOps pugui revisar exactament quins servidors es crearan o esborraran.

3. Fase de Desplegament (*Apply*):

- Un cop es fusiona el codi a la branca principal (*main*), la pipeline s'executa de nou i llança l'ordre de creació real (ex. `terraform apply`) al núvol. En entorns de producció, se sol col·locar una **tanca d'aprovació manual** (*Manual Gate*), on un enginyer sènior ha de prémer un botó per a autoritzar el desplegament definitiu.

3.2. Gestió de credencials segures del proveïdor

Perquè el robot de la pipeline (ex. *GitHub Actions*) pugui crear servidors a AWS, Azure o GCP, necessita permisos d'administrador de la infraestructura. Emmagatzemar claus d'accés permanents (com un parell d'usuari i contrasenya o claus *Access Key / Secret Key* d'AWS) dins de la configuració de la pipeline és una mala pràctica d'alt risc. Si la plataforma de Git és atacada, les claus permanents queden exposades.

Per a evitar-ho, els entorns DevOps moderns utilitzen dues estratègies de seguretat avançada:

A. Autenticació federada mitjançant OIDC (OpenID Connect) — L'estàndard recomanat

En lloc d'utilitzar claus d'accés fixes, es configura una relació de confiança mútua i directa entre el proveïdor de Git (GitHub) i el proveïdor de núvol (AWS/Azure/GCP).

- **Com funciona:**

Quan la pipeline comença a funcionar, GitHub emet un token d'identitat digital de curta durada. La pipeline presenta aquest token a l'API d'AWS. AWS el valida, comprova que prové del repositori oficial de l'empresa i li atorga un **rol temporal** (que caduca en pocs minuts) amb els permisos justos per a fer eixe desplegament.

- **Avantatge: Zero secrets guardats.** No hi ha cap contrasenya escrita enlloc, eliminant per complet el risc de fuites de credencials.

B. Ús de Secrets Xifrats de la Plataforma

Si l'autenticació OIDC no és possible, les credencials s'han d'allotjar obligatòriament a l'apartat de **Secrets** de la plataforma de CI/CD (ex. *GitHub Encrypted Secrets* o les variables protegides de *GitLab*).

- Les claus viatgen sempre xifrades i s'injecten en la memòria RAM del robot en temps d'execució com a variables d'entorn.
- La plataforma s'encarrega d'emascarar automàticament els valors en els registres de text (*logs*), evitant que qualsevol treballador pugui visualitzar la contrasenya en pantalla.

4. Polítiques de seguretat digital en entorns

La seguretat en els entorns de desplegament requereix passar de les definicions conceptuals a la configuració tècnica real del sistema. Aquest punt descriu com l'equip de DevOps implementa les restriccions i els perímetres de seguretat per blindar l'empresa davant d'amenaces en quatre capes crítiques:

1. Rols d'usuari i control d'accessos perimetrals

El perímetre d'accés ja no és només una barrera de xarxa, sinó que es defineix a través de la identitat.

- **Rols de xarxa perimetral (Bastion / Jump Hosts):** Perquè els enginyers puguin fer tasques de manteniment a les subxarxes privades de producció, es prohibeix l'accés directe. Es configura tècnicament un servidor intermediari fortificat anomenat *Bastion*. Els usuaris s'autentiquen primer en aquest punt mitjançant certificats SSH únics i **MFA (Autenticació Multifactor)**, i és des d'aquí des d'on s'enllaça amb l'entorn intern.
- **Control d'Accessos Basat en Rols (RBAC):** S'aplica programant polítiques en sistemes IAM on els permisos van lligats al rol del treballador en eixe moment concret, i no a la seva persona, assegurant la revocació automàtica quan canvia de projecte.

2. Permisos d'execució del codi

Un cop el codi s'està executant als servidors, s'ha d'evitar que una aplicació web compromesa pugui controlar tota la màquina virtual o el clúster de contenidors.

- **Execució sense privilegis (Non-root execution):** És una regla obligatòria configurar els fitxers (com els *Dockerfile*) perquè l'aplicació web s'executi sota un usuari del sistema

operatiu completament limitat (ex: USER node o USER appuser) en lloc de l'usuari root. Si un atacant troba una esquadra al codi de la web, no podrà instal·lar programari maliciós ni esborrar el sistema operatiu perquè el procés no té permissos per a fer-ho.

- **Sistemes de fitxers de només lectura:** Es configura l'entorn d'execució (el motor de contenidors) perquè el disc core on està el codi sigui de **només lectura** (read-only root filesystem). L'aplicació pot funcionar, però ningú pot modificar els fitxers existents en temps d'execució.

3. Polítiques d'emmagatzematge segur

Les dades en repòs són l'objectiu principal dels ciberatacs. Les polítiques tècniques s'apliquen de forma estricta:

- **Xifratge global forçós:** Es programen polítiques a la IaC (Terraform) que impedeixen la creació de qualsevol volum de disc o repositori d'objectes (com AWS S3) si no tenen activat el xifratge de dades en repòs mitjançant claus gestionades (**AES-256** o serveis com AWS KMS).
- **Bloqueig d'accés públic natiu:** Per evitar errors humans, els proveïdors de núvol permeten activar regles a nivell de compte (*Block Public Access*) que bloquegen qualsevol intent de fer que un repositori de dades sigui accessible des d'Internet de forma pública, actuant com una xarxa de seguretat global.

4. Seguretat en crides a APIs

Les aplicacions modernes es comuniquen contínuament mitjançant interfícies de programació (APIs). Protegir aquests intercanvis és vital per evitar la intercepció de dades:

- **Xifratge en trànsit obligatori (TLS):** Totes les crides entre microserveis o cap a APIs externes s'han de fer obligatòriament sota protocols segurs (HTTPS / **TLS 1.3**). Es configuren polítiques de xarxa que rebutgen automàticament qualsevol trànsit que viatgi en text pla (HTTP).
- **Autenticació mitjançant tokens de curta durada:** Es prohibeix l'ús de claus d'API estàtiques i infinites al codi. Es fan servir sistemes d'intercanvi de tokens dinàmics (com **OAuth 2.0** o *JSON Web Tokens - JWT*), que caduquen al cap de pocs minuts, i s'implementen mecanismes de limitació de taxa (*Rate Limiting*) als tallafocs d'aplicacions web (WAF) per evitar atacs de denegació de servei (DoS) o intents de força bruta.

Resum Tècnic de les Capes de Seguretat Digital

Capa de l'Entorn	Mesura Tècnica de Configuració	Objectiu Principal
Accés Perimetral	Servidor <i>Bastion</i> + MFA + Polítiques IAM RBAC.	Impedir entrades no autoritzades a la xarxa privada.
Execució del Codi	Usuaris non-root en contenidors i sistemes <u>read-only</u> .	Minimitzar l'impacte si la web és hackejada.
Emmagatzematge	Xifratge KMS obligatori i bloqueig públic global.	Evitar fuites de dades i auditar el contingut en repòs.

5. Desplegament automatitzat d'infraestructura

El desplegament automatitzat d'infraestructura és la fase d'execució real on els plànols digitals (el codi escrit en Terraform, CloudFormation, etc.) es converteixen en recursos físics i virtuals actius dins d'un proveïdor de núvol. Aquest procés substitueix la interacció manual per un flux controlat que connecta l'orquestrador de la IaC amb les APIs del Cloud. A continuació, es desenvolupen els aspectes tècnics d'aquesta fase d'execució:

1. El mecanisme d'execució: La connexió amb l'API del Cloud

Les plataformes de núvol moderns (AWS, Azure, GCP) són, en essència, grans interfícies de programació d'aplicacions (**APIs**). Quan s'executa un desplegament automatitzat, l'eina de IaC realitza els següents passos interns:

- **Traducció a peticions HTTP:** L'eina agafa el codi (per exemple, un bloc `resource "aws_instance"` de Terraform) i el tradueix en una crida d'API real (una petició HTTP POST cap a l'endpoint d'AWS).
- **Autenticació de la sessió:** La comanda d'execució injecta el token temporal obtingut de forma segura (mitjançant OIDC o secrets xifrats) en la capçalera de la petició per autoritzar l'acció.
- **Resposta de la plataforma:** El proveïdor de núvol valida la petició, reserva els recursos de maquinari real en segons i respon a l'eina amb les dades del nou recurs (ID de la instància, adreça IP assignada, etc.).

2. Ordre d'execució i resolució de dependències

Un dels avantatges del model declaratiu en l'execució real és la **creació del gràfic de dependències** (*Dependency Graph*). L'eina d'IaC analitza tot el fitxer de configuració abans d'enviar res al núvol per determinar l'ordre correcte de creació:

- **Dependències implícites:** Si el codi defineix una base de dades que s'ha d'ubicar dins d'una subxarxa concreta, l'eina sap de forma intel·ligent que primer ha d'executar el desplegament de la subxarxa i esperar que el núvol li torni la confirmació abans d'intentar aixecar la base de dades.
- **Execució en paral·lel:** Tots els recursos que no depenguin els uns dels altres (per exemple, deu discos durs virtuals independents) es despleguen de manera simultània per minimitzar el temps total d'espera.
-

3. El registre de la realitat: El fitxer d'Estat (*State File*)

Durant l'execució d'eines com Terraform, es genera o s'actualitza un component crític: el **fitxer d'estat** (`.tfstate`).

- **Què és?** És un document en format JSON que actua com la veritat absoluta del sistema. Registra la correspondència exacta entre els recursos escrits en codi i els IDs reals existents al núvol.
- **Bloqueig d'estat (*State Locking*):** Quan la Pipeline comença l'execució real, bloqueja aquest fitxer en un magatzem remot segur (com AWS S3 amb DynamoDB). Això impedeix

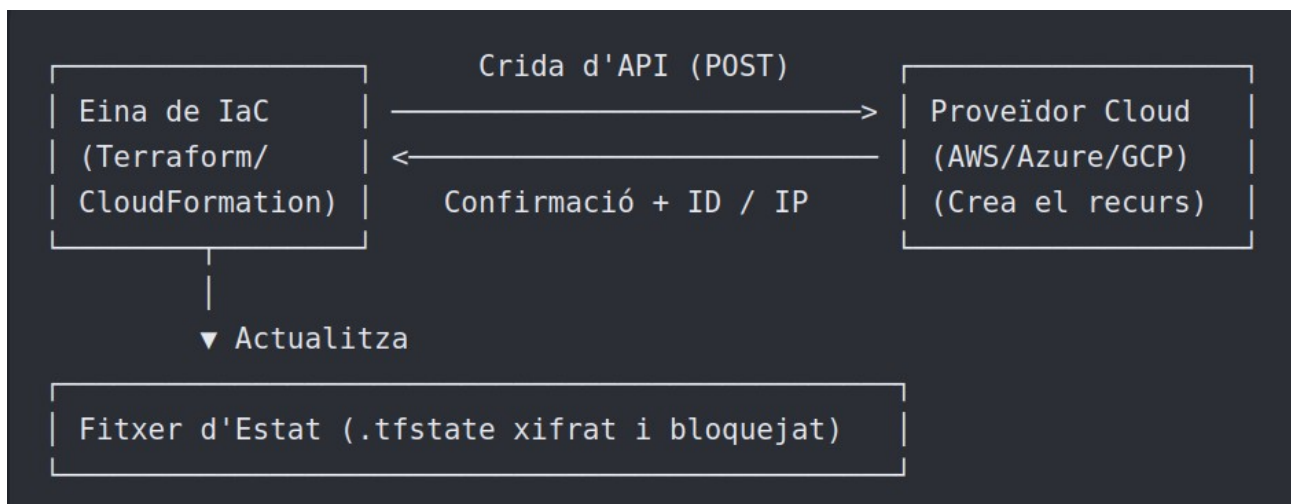
que una altra execució simultània (per exemple, dos enginyers llançant la pipeline alhora) intenti modificar el mateix maquinari, evitant conflictes i corrupció de la xarxa.

4. Validació Post-Desplegament i tancament del flux

L'execució real no acaba quan el proveïdor de núvol diu "recurs creat"; cal verificar-ne el funcionament correcte:

- **Proves de fum (*Smoke Tests*):** La Pipeline d'infraestructura sol incloure un pas final que realitza comprovacions bàsiques directes sobre l'entorn tot just aixecat (per exemple, fer una crida *ping* o *curl* a la IP del servidor web creat per comprovar que respon un codi 200 OK).
- **Injecció d'Outputs (Resultats):** L'eina extreu dades rellevants de l'execució (com la URL del balancejador de càrrega o les claus de connexió) i les exporta com a paràmetres per a la següent Pipeline, que serà l'encarregada de desplegar el codi de l'aplicació sobre aquest maquinari net.

Resum del procés d'execució automatitzada



6. Validació de l'entorn i inspecció postdesplegament

La validació de l'entorn i la inspecció postdesplegament és l'última línia de defensa en el cicle d'automatització d'infraestructures (IaC). Un cop el proveïdor de núvol confirma que ha creat els recursos, no podem assumir que tot funciona correctament; és imprescindible executar proves automatitzades per certificar que el maquinari virtual és accessible, està ben configurat i es comunica de forma segura abans de donar el desplegament per bo.

6.1. Tipologies de proves postdesplegament

Un cop finalitzada l'execució del codi d'IaC, la pipeline activa una sèrie de tests agrupats per objectiu tècnic:

- **Proves de connectivitat i xarxa (*Smoke Tests*):** Realitzen comprovacions bàsiques a nivell de transport per verificar que els tallafocs (*Security Groups*) i encaminadors estan ben configurats.

- *Exemple:* Comprovar mitjançant comandes automatitzades (ping o nc -z) que el port 22 (SSH) és accessible només des de la xarxa interna de l'empresa, o que el port 443 (HTTPS) respon correctament a peticions des d'Internet.
- **Proves de salut de l'aplicació (Health Checks):** Es realitzen crides HTTP dirigides a rutes específiques de diagnòstic (com <https://empresa.com>). El sistema d'inspecció valida que el servidor respon un codi d'estat **200 OK** i que el temps de resposta (latència) es troba dins dels paràmetres esperats (per exemple, inferior a 200 ms).
- **Verificació de configuració profunda (Compliance Testing):** Eines d'auditoria verifiquen que els servidors compleixen els requisits de seguretat establerts en les polítiques de l'empresa (per exemple, comprovar que els discos durs estan realment xifrats amb la clau AES-256 correcta).

6.2. Eines d'automatització per a la inspecció

L'equip de DevOps no fa aquestes comprovacions entrant a les màquines a mà. S'utilitzen frameworks de codi que llegeixen la infraestructura com si fos programari:

- **InSpec / Terratest:** Són eines que permeten escriure proves utilitzant llenguatges de programació (com Ruby o Go).
- *Exemple de lògica de test:* Un script de Terratest pot extreure de forma automàtica la IP del servidor que Terraform acaba de crear, realitzar una crida HTTP a eixa IP i validar si el contingut de la pàgina web és el correcte. Si el test falla, Terratest ordena la pipeline aturar el procés.
- **Proves nates dels balancejadors (Load Balancer Health Checks):** Els propis components del núvol s'encarreguen d'una inspecció contínua. El balancejador de càrrega envia peticions de control cada pocs segons a les màquines virtuals del grup. Si una màquina triga massa a respondre, el sistema la marca com a "insalubre" (*Unhealthy*) i desvia tot el trànsit cap a altres servidors sans per evitar que els usuaris rebin errors.

6.3. Flux de gestió d'errors (Rollback Automatitzat)

Què passa si la inspecció postdesplegament detecta que un dels recursos no opera sota els paràmetres esperats? Una pipeline de DevOps ben dissenyada ha de gestionar la fallada de forma controlada:

1. **Aturada del flux:** Es cancel·la el desplegament del codi de l'aplicació per evitar enviar programari a un maquinari defectuós.
2. **Notificació immediata:** S'envia una alerta crítica als canals de l'equip de guàrdia (Slack, Microsoft Teams o PagerDuty) detallant quin test de validació ha fallat.
3. **Estratègia de Retorn (Rollback):** Depenent de la configuració del projecte, l'eina de IaC pot executar una comanda automàtica per a revertir la infraestructura a l'últim estat estable conegut (restaurar el fitxer d'estat `.tfstate` anterior), garantint que l'entorn de producció mai es quedi en un estat inestable o a mig construir.

--ooOOoo--

Tema 7: Gestió i monitorització en DevOps



1. Eines de gestió de la configuració i automatització
2. Disseny de fluxos de manteniment i configuració
3. Proves i verificació dels fluxos de manteniment
4. Parametrització i polítiques de seguretat per a contenidors
5. Monitorització, mètriques e informes
6. Documentació tècnica de la infraestructura
7. Automatització basada en esdeveniments

1. Eines de gestió de la configuració i automatització

La gestió de la configuració i l'automatització representa la capa que actua just després de la creació de la infraestructura. Mentre que eines com Terraform s'encarreguen d'aixecar el "ferro" o la màquina virtual buida (IaC), les eines de gestió de la configuració s'ocupen de l'interior de la màquina: instal·lar paquets de programari, aplicar configuracions de seguretat, gestionar usuaris del sistema operatiu i desplegar fitxers d'aplicació de manera unificada a milers de servidors simultàniament.

1.1. El concepte d'automatització de la configuració

En l'administració de sistemes tradicional, un enginyer entrava per SSH a cada servidor per a executar comandes com `apt-get install nginx` o modificar l'arxiu `nginx.conf` a mà. Aquest procés manual és lent, propens a l'error humà i provoca la derivació de la configuració (*configuration drift*), on amb el temps cada servidor acaba configurat de manera diferent.

Les eines de gestió de la configuració resolen aquest problema mitjançant:

- **Estandardització:** Totes les instal·lacions es defineixen en fitxers de codi centralitzats.
- **Idempotència forçada:** Si executes l'eina en un servidor on el programari ja està instal·lat i configurat correctament, l'eina no farà res ni reiniciarà el servei per error; només actuarà si detecta que la realitat no coincideix amb el codi.

1.2. Models d'arquitectura: Amb agent (Pull) vs. Sense agent (Push)

El mercat es divideix en dues grans filosofies de funcionament segons com es comuniquen l'eina central i els servidors destí:

ARQUITECTURES DE GESTIÓ	
MODEL PUSH (Ansible) L'ordinador central es connecta per SSH i "empeny" les configuracions cap als servidors de forma directa.	MODEL PULL (Puppet) Cada servidor té un agent instal·lat que "demana" les ordres al servidor central.

A. Model Push / Sense Agent (Exemple de referència: Ansible)

L'eina central es connecta directament als servidors de destí utilitzant els protocols de comunicació nadius del sistema operatiu (com **SSH** per a Linux o **WinRM** per a Windows).

- **Com funciona:** Ansible llegeix un fitxer d'inventari (on estan les IPs dels servidors) i executa un llibre de receptes o **Playbook** (escrit en YAML).
- **Avantatge:** És molt lleuger i ràpid d'implementar perquè **no cal instal·lar cap programari de control (agent)** als servidors de destí.

B. Model Pull / Amb Agent (Exemple de referència: Puppet / Chef)

Cada servidor de la infraestructura té un petit programari instal·lat en segon pla anomenat **Agent**.

- **Com funciona:** Cada cert temps (per exemple, cada 30 minuts), l'agent es connecta al servidor central (*Puppet Master*), es descarrega el codi de configuració que li pertoca i l'aplica de forma local.
- **Avantatge:** És excel·lent per a infraestructures massives de milers de servidors, ja que detecta de manera autònoma si un usuari ha modificat un fitxer a mà localment i el reverteix immediatament a l'estat oficial.

1.3. Ús pràctic: Exemple conceptual d'un Playbook d'Ansible

Per a instal·lar i activar un servidor web, un enginyer DevOps escriu un fitxer YAML declaratiu on detalla les tasques que s'han d'executar a la infraestructura d'estat:

yaml

```
- name: Configurar servidors web de l'empresa
  hosts: servidors_web # Grup d'IPs definit a l'inventari
  become: yes          # Executa com a superusuari (root)

  tasks:
    - name: 1. Assegurar que el servidor Nginx està instal·lat
      apt:
        name: nginx
        state: present # Si no hi és, l'instal·la

    - name: 2. Copiar l'arxiu de configuració corporatiu
      copy:
        src: /ficheros_locales/nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Reiniciar Nginx # Si l'arxiu canvia, llança l'acció inferior

  handlers:
    - name: Reiniciar Nginx
      service:

name: nginx
  state: restarted
```

2. Disseny de fluxos de manteniment i configuració

Els fluxos de manteniment i configuració automatitzats són essencials per garantir la seguretat, l'estabilitat i la consistència dels sistemes en entorns de producció. A

continuació, es presenta el disseny detallat d'aquests fluxos utilitzant pràctiques modernes de **GitOps** i eines d'Infraestructura com a Codi (IaC).

2.1. Flux d'Automatització de Pedaços (Patch Management)

Aquest flux assegura que els sistemes operatius i les dependències crítiques rebin les actualitzacions de seguretat de forma regular sense intervenció manual directa, minimitzant el temps d'exposició a vulnerabilitats.

- **Detecció de vulnerabilitats:** Un agent (ex. *Wazuh*, *Trivy* o *Amazon Inspector*) escaneja els servidors de forma diària.
- **Generació de l'alerta:** Si es detecta un pedaç crític, el sistema obre automàticament un tiquet a la cua de manteniment.
- **Aplicació en entorn de Staging:** Un *playbook* d'**Ansible** s'executa automàticament per instal·lar els pedaços en un entorn de proves clonat de producció.
- **Proves automatitzades:** S'executen tests de fum (*smoke tests*) i de regressió per verificar que l'actualització no trenca cap servei.
- **Desplegament en Producció Canari:** Si els tests funcionen, s'apliquen els pedaços a un percentatge reduït de nodes de producció (10%).
- **Desplegament Complet:** Després de 24 hores sense alertes mètriques, els pedaços s'apliquen a la resta de la infraestructura de forma seqüencial (*rolling updates*).

2.2. Flux d'Actualització de Serveis (CI/CD i Rolling Deployments)

Dissenyat per actualitzar les aplicacions i els serveis interns reduint el temps d'inactivitat a zero (*Zero-Downtime*).

[Codi font (Git)] → [Pipeline CI: Test i Build] → [Registre d'Imatges] →
[Desplegament Progressiu] → [Verificació de Mètriques]

- **Disparador (Trigger):** El desenvolupador fa un *merge* a la branca main o crea una nova etiqueta de versió (*tag*).
- **Construcció d'artefactes:** El pipeline de **GitHub Actions** o **GitLab CI** construeix la nova imatge de Docker i la puja al registre privat.
- **Actualització de l'estat desitjat:** El pipeline modifica el fitxer de manifest de **Kubernetes** o el repositori de GitOps (controlat per **ArgoCD** o **Flux**).
- **Desplegament Progressiu (Rolling Update):**
 - Es creen els nous pods amb la versió actualitzada.
 - Els *health checks* (*liveness* i *readiness probes*) verifiquen que el nou servei respon correctament.
 - Es redirigeix el trànsit progressivament i s'eliminen els pods antics.
- **Rollback automàtic:** Si el percentatge d'errors HTTP 5xx augmenta durant els primers 5 minuts, el sistema reverteix el canvi immediatament a la versió anterior.

2.3. Flux de Consistència del Programari del Sistema (Configuration Management)

Aquest flux garanteix que tots els servidors d'un mateix clúster mantinguin exactament la mateixa configuració, evitant la "desviació de configuració" (*configuration drift*).

- **Definició de l'estat desitjat:** Tota la configuració del sistema (usuaris, paquets instal·lats, regles de tallafocs, fitxers /etc) es defineix com a codi en un repositori Git central.
- **Auditoria contínua:** Un agent de **SaltStack** o **Puppet** s'executa cada 30 minuts a cada node.
- **Detecció de desviacions:** L'agent compara l'estat actual del servidor amb l'estat definit a Git.
- **Correcció automàtica (Remediation):** Si un administrador ha modificat manualment un fitxer a producció, l'agent sobreescriu el canvi per retornar al model correcte de Git.
- **Notificació:** Qualsevol acció de correcció es reporta a un canal de Slack o Teams per a la seva supervisió.

2.4. Beneficis del Model Proposat

- **Traçabilitat total:** Qualsevol canvi en el programari del sistema queda registrat a Git (Qui, Què i Quan).
- **Repetibilitat:** La infraestructura es pot recrear des de zero en minuts en cas de desastre catastròfic.
- **Seguretat reforçada:** S'elimina la necessitat que els enginyers accedeixin per SSH directament als servidors de producció.

3. Proves i verificació dels fluxos de manteniment

Les proves i verificació dels fluxos de manteniment són un pilar fonamental per garantir la resiliència del sistema. L'objectiu principal és comprovar que les automatitzacions de recuperació (auto-healing) funcionen correctament davant d'incidents reals sense necessitat d'intervenció humana.

A continuació, es detalla la metodologia per dissenyar la simulació de fallades i validar-ne les accions correctives.

3.1. Estratègia de Simulació de Fallades (Chaos Engineering)

Per validar els fluxos automatitzats, cal injectar errors de manera controlada als entorns de proves (o producció, si es disposa de la maduresa necessària).

- **Fallades d'Infraestructura:**
 - *Simulació:* Aturada sobtada d'una instància de servidor o contenidor (pod).
 - *Resultat esperat:* El balancejador de càrrega redirigeix el trànsit i el sistema d'orquestració (ex. Kubernetes) aixeca una nova instància automàticament.
- **Saturació de Recursos:**
 - *Simulació:* Injecció d'estrès de CPU al 99% o omplir el disc de manera artificial mitjançant scripts.

- *Resultat esperat:* Alerta de monitoratge disparada i activació de l'escalat automàtic (*auto-scaling*) o neteja automàtica de brossa (*logs* antics).
- **Degradació de la Xarxa:**
- *Simulació:* Introducció de latència artificial (ex. 5000ms) o pèrdua de paquets en la comunicació amb la base de dades.
- *Resultat esperat:* Activació de mecanismes de *Circuit Breaker*, degradació elegant del servei (motius de fallada clars) i reintents automàtics exponencialment fets.

3.2. Matriu de Validació d'Accions Automatitzades

Es recomana seguir una matriu de proves per documentar que cada acció corregeix el problema de la manera esperada:

Incident Simulat	Alerta Disparada	Acció Automatitzada Executada	Estat Final Esperat	Temps de Recuperació (RTO)
Caiguda del servei API	HTTP 5xx Error Rate > 5%	Reinici del contenidor / servei	Servei actiu (HTTP 200)	< 30 segons
Disc al 85% de capacitat	DiskSpaceRunningLow	Script de purga de <i>logs</i> temporals	Espai lliure > 30%	< 2 minuts
Bloqueig de Base de Dades	DB_Deadlock_Detected	Matar la connexió causant / <i>Failover</i> a la rèplica	DB operativa i fluida	< 1 minut

3.3. Fases del Procés de Verificació

1. **Definició de l'estat estacionari:** Determinar quines són les mètriques normals del sistema (mètriques de salut o *KPIs*).
2. **Injecció de l'error:** Executar la simulació planificada (manualment o amb eines com *Chaos Mesh* o *Gremlin*).
3. **Monitoratge de la resposta:** Verificar que l'alarma es genera en el temps previst i que el llibre de ruta automatitzat (*runbook*) s'executa.
4. **Validació del retorn a la normalitat:** Comprovar que el sistema recupera el seu estat estacionari sense corrupció de dades ni pèrdua de servei.

4. Parametrització i polítiques de seguretat per a contenidors

La parametrització i definició de polítiques de seguretat en contenidors és essencial para protegir la infraestructura i garantir la resiliència del sistema. Una configuració avançada en les eines de desplegament (com Kubernetes o Docker Compose) evita que un contenidor compromès o defectuós afecti la resta de l'ecosistema.

A continuació, es detallen les configuracions clau estructurades en els tres blocs sol·licitats.

4.1. Gestió i Límits de Consum de CPU i Memòria RAM

La restricció de recursos evita que un contenidor consumeixi tota la capacitat del node (*Noisy Neighbor effect*). En Kubernetes, es configuren mitjançant dos paràmetres crítics:

- **requests (Garantia mínima):** Els recursos que el contenidor té assegurats des del moment del seu llançament. L'orquestrador utilitza aquest valor per decidir en quin node ubicar el contenidor.
- **limits (Sostre màxim):** El límit absolut que el contenidor no pot superar.
- **CPU:** Si es supera el límit, el sistema **alenteix (throttles)** el contenidor, però no el mata.
- **RAM:** Si es supera el límit, el nucli del sistema operatiu actua i **mata el contenidor immediatament** amb un error de falta de memòria (**OOMKilled**).

Yaml

```
# Exemple de configuració en Kubernetes
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"    # 0.25 vCPU
  limits:
    memory: "512Mi"
    cpu: "500m"    # 0.50 vCPU
Utilitza el codi amb precaució.
```

4.2. Regles d'Aïllament i Polítiques de Seguretat

L'aïllament assegura que els contenidors s'executin amb els privilegis mínims necessaris, reduint la superfície d'atac.

- **Polítiques de Seguretat del Pod (Security Context):**
- **runAsNonRoot: true:** Obliga a l'aplicació a executar-se amb un usuari sense privilegis d'administrador.
- **allowPrivilegeEscalation: false:** Evita que els processos fills puguin obtenir més privilegis que el procés pare.
- **readOnlyRootFilesystem: true:** Bloqueja l'escriptura al sistema de fitxers del contenidor. Qualsevol escriptura s'ha de fer en volums temporals (emptyDir), evitant la injecció de codi maliciós persistent.
- **Polítiques de Xarxa (Network Policies):**
- Per defecte, tots els contenidors es poden comunicar entre ells. Cal aplicar una política de **Deny-All** (denegar tot) i obrir explícitament només els fluxos necessaris.
- *Exemple:* Permetre que el *Frontend* parli amb el *Backend*, però bloquejar totalment que el *Frontend* tingui accés directe a la *Base de Dades*.

4.3. Nivells de Servei (SLA) i Qualitat de Servei (QoS)

L'orquestrador gestiona la prioritat dels contenidors en moments d'estrès de la màquina basant-se en les definicions de recursos anteriors. Això es tradueix en tres classes de **Qualitat de Servei (QoS)** que defineixen directament l'SLA intern del servei:

Classe de QoS	Configuració requerida	Comportament davant d'estrès (SLA)
Guaranteed (Alta Prioritat)	Els valors de <u>requests</u> i <u>limits</u> són exactament iguals (tant en CPU eliminats. Ideal per a serveis crítics o bases de	Són els darrers contenidors en ser expulsats o

	com en RAM).	dades.
Burstable (<i>Prioritat Mitjana</i>)	El valor de <u>requests</u> és menor que el de <u>limits</u> .	El contenidor pot demanar més recursos si n'hi ha de disponibles, però corre el risc de ser expulsat si el node s'omple.
BestEffort (<i>Baixa Prioritat</i>)	No es defineix cap límit ni petició (<u>requests</u> / <u>limits</u>).	Utilitzen els recursos sobrants de la màquina. Són els primers a ser eliminats immediatament si el sistema es queda sense memòria.

5. Monitorització, mètriques e informes

La monitorització, el control de mètriques i l'elaboració d'informes configuren el sistema nerviós central de qualsevol infraestructura moderna. No es tracta només de saber si el sistema està actiu o caigut, sinó d'entendre'n el comportament profund per anticipar-se a les fallades i optimitzar els costos.

A continuació, es detallen els tres pilars per implementar una estratègia de telemetria i capacitat eficaç.

5.1. Captura de Telemetria dels Entorns

La recollida de dades s'ha de fer en temps real mitjançant agents (com *Prometheus Node Exporter*, *Datadog Agent* o *Telegraf*) organitzats en quatre capes clau:

- **Mètriques d'Infraestructura i Sistema (Mínims vitals):**
- **CPU:** Percentatge d'ús global i per nucli, així com mètrics d'espera d'I/O (*I/O wait*).
- **Memòria:** Percentatge d'ús de la RAM, memòria cau (*cache*) i memòria d'intercanvi (*swap*).
- **Xarxa:** Ample de banda utilitzat (bytes d'entrada/sortida), paquets perduts i errors de transmissió.
- **Mètriques de Disc i Emmagatzematge:**
- Espai lliure en volum i taxa d'**IOPS** (operacions de lectura/escriptura per segon), crítica per a bases de dades.
- **Mètriques d'Aplicació (Els 4 Senyals Daurats de Google):**
- **Latència:** Temps que triga a respondre una petició (mesurat en els percentils **p50, p95 i p99**).
- **Trànsit:** Volum de demanda (ex. peticions HTTP per segon o missatges a la cua).
- **Errors:** Taxa de peticions que fallen (ex. codis HTTP 5xx o excepcions de codi).
- **Saturació:** Quant de "plena" està l'aplicació (ex. percentatge de fils de memòria o connexions de la BD ocupades).

5.2. Elaboració d'Informes de Rendiment

Els informes transformen les dades brutes de la telemetria en coneixement de negoci i tècnic. S'acostumen a dividir en dos formats:

- **Taulers de control operatius en temps real (Dashboards com Grafana):** Destinats a l'equip d'operacions (SRE/DevOps) per reaccionar ràpidament durant un incident. Mostren l'estat actual dels darrers 30 minuts o 24 hores.

- **Informes analítics periòdics (Setmanals/Mensuals):** Destinats a la direcció tècnica (CTO/Managers). Se centren en:
- **Disponibilitat real:** Percentatge de temps d'activitat de l'entorn mesurat contra l'SLA (ex. 99.9% d'Uptime).
- **Tendències de consum:** Identificació de colls d'ampolla habituals o moments del dia amb pics de trànsit recurrents.
- **Anàlisi d'incidents:** Resum d'alertes disparades, causes arrel (*Root Cause Analysis*) i temps mitjà de resolució (MTTR).

5.3. Planificació de Capacitat (*Capacity Planning*) i Escala

La planificació de capacitat utilitza les dades històriques acumulades per projectar les necessitats d'infraestructura futures, evitant tant la saturació del sistema com el malbaratament de pressupost (*over-provisioning*).

- **Anàlisi de Tendència Lineal:** Si el consum de disc de la base de dades creix a un ritme constant de **50 GB al mes**, es pot calcular amb exactitud quin dia el disc arribarà al seu límit de seguretat (ex. 80%) per ampliar-lo abans que passi.
- **Correlació amb Mètriques de Negoci:** Vincular les mètriques tècniques amb el comportament del negoci (ex. "*Cada 1.000 nous usuaris registrats actius, el consum de CPU del backend augmenta un 4%*").
- **Proves de Càrrega i Estrès (*Load Testing*):** Simular escenaris de trànsit massiu (com un *Black Friday* o una campanya de màrqueting) per determinar el punt de trencament del sistema i configurar correctament les regles d'escalat automàtic (*Auto-scaling*).

6. Documentació tècnica de la infraestructura

La documentació tècnica de la infraestructura gestionada directament dins dels propis repositoris Git es coneix com a Docs-as-Code (Documentació com a Codi). Aquesta pràctica garanteix que el codi de la infraestructura (IaC), les configuracions i la seva documentació evolucionin conjuntament, utilitzant el mateix sistema de control de versions, branques (*branches*) i revisions de codi (*Pull Requests*).

A continuació, es detalla l'estructura i el contingut clau per documentar correctament la infraestructura en Git.

6.1. Estructura de Fitxers Recomanada en el Repositori

La documentació ha de viure al costat de les configuracions tècniques. Es recomana crear una carpeta /docs o col·locar fitxers Markdown (.md) estratègics a l'arrel del repositori:

la-teua-infraestructura/	
├─ .github/workflows/	# Fluxos de CI/CD
├─ deploy/	# Fitxers de configuració (K8s, Docker
Compose, Terraform)	
├─ docs/	# Carpeta de documentació centralitzada
│ ├─ runbooks.md	# Manuals d'operacions d'emergència
│ └─ architecture.md	# Diagrames i descripció de l'arquitectura
├─ Dockerfile	# Definició de la imatge del contenidor
├─ README.md	# Guia d'ús principal (Punt d'entrada)
└─ RELEASES.md	# Historial de versions de la
infraestructura	

6.2. Contingut dels Manuals i Guies Requerits

Per a complir amb els estàndards tècnics, la documentació s'ha de dividir en tres blocs clau:

A. Guies d'Ús i Arquitectura ([README.md](#))

És la primera pantalla que veu un desenvolupador o administrador de sistemes. Ha de ser clara i directa.

- **Descripció de l'entorn:** Què fa aquesta infraestructura i quins serveis allotja.
- **Prerequisits:** Eines que cal tenir instal·lades localment (ex. [docker](#), [kubectl](#), [aws-cli](#)).
- **Guia de llançament ràpid (*Quickstart*):** Les comandes exactes per aixecar l'entorn local de proves en menys de 5 minuts.
- **Variables d'entorn:** Taula explicativa amb els paràmetres de configuració ([.env.example](#)) i quins són obligatoris.

B. Configuracions de Desplegament de Codi

Explica com es mou el codi des de l'ordinador del desenvolupador fins a l'entorn de producció.

- **Estratègia de branques (GitFlow / Trunk-based):** Explicació de quina branca (ex. [main](#) o [develop](#)) dispara el desplegament a cada entorn.
- **Flux de CI/CD (Continuous Integration / Continuous Deployment):** Documentació dels passos automatitzats (passar tests, compilar, escanejar seguretat i desplegar).
- **Gestió de Secrets:** Instruccions (sense exposar contrasenyes reals) de com es gestionen les credencials del sistema (ex. utilitzant *GitHub Secrets*, *AWS Secrets Manager* o *HashiCorp Vault*).

C. Manual d'Imatges de Contenidors

Documentació específica sobre com es construeixen, optimitzen i emmagatzemen les imatges de Docker.

- **Instruccions de compilació:** Comandes per crear la imatge manualment si fos necessari (`docker build -t la-meva-app:1.0 .`).
- **Política d'etiquetatge (*Tagging*):** Com es gestionen les versions de les imatges (ex. utilitzar el codi *hash* del commit de Git o versions semàntiques *Semantic Versioning* com `v1.2.3`, evitant l'ús del tag generíc `latest` en producció).
- **Registre de contenidors (*Registry*):** On s'emmagatzemen les imatges (ex. *Docker Hub*, *GitHub Container Registry*, *AWS ECR*) i com autenticar-se per descarregar-les.

6.3. Bones Pràctiques per a la Documentació en Git

- **Llenguatge Markdown:** Utilitzar codi net, blocs de codi (````bash`) per a les comandes i taules per a les configuracions. Això facilita la lectura des de plataformes com GitHub o GitLab.
- **Automatització (Linters):** Validar que els enllaços de la documentació no estiguin trencats utilitzant eines en el CI/CD com `markdownlint`.
- **Actualització obligatòria:** Aplicar la norma de l'equip: *"Si un canvi de codi o configuració modifica la manera com funciona el sistema, la Pull Request no s'aprova fins que la documentació estigui actualitzada"*.

7. Automatització basada en esdeveniments

L'automatització basada en esdeveniments (Event-Driven Automation) transforma el monitoratge passiu en un sistema actiu d'autoreparació (auto-healing). En lloc de limitar-se a enviar una notificació per correu o Slack quan un sistema falla, l'entorn detecta l'esdeveniment anòmal i executa immediatament una acció correctiva programada, reduint el Temps Mitjà de Resolució (**MTTR**) a zero i minimitzant la intervenció humana.

A continuació, es detalla el funcionament del bucle de control i els escenaris de configuració clau.

7.1. El Bucle de l'Autoreparació (MAPE-K)

Qualsevol flux d'autoreparació es basa en un cicle tancat de quatre fases connectades en temps real:

1. **Monitoritzar (Event):** L'agent de telemetria recull una mètrica fora de rang (ex. *Prometheus* detecta una fallada de sondeig o l'orquestrador veu un contenidor caigut).
2. **Analitzar (Trigger):** El motor de regles avalua si es compleix la condició d'alerta (ex. *Prometheus Alertmanager* processa l'esdeveniment).
3. **Planificar (Action/Runbook):** Es tradueix l'alerta en una petició d'execució dirigida a una eina d'automatització (ex. *Ansible Rulebook*, *AWS Lambda* o un script d'orquestració).

4. **Executar (Remediation):** S'aplica l'acció tècnica sobre la infraestructura i es valida si el sistema ha tornat a l'estat original de salut.

7.2. Configuració d'Alertes i Desencadenants (Triggers) Comuns

A continuació es detallen els tres patrons d'autoreparació més utilitzats en la gestió d'infraestructures modernes:

A. Reinici automàtic de serveis bloquejats (*Process/Container Crash*)

- **Esdeveniment:** Un pod o contenidor deixa de respondre a les proves de vida (*Liveness Probes*) o retorna errors de connexió consecutius.
- **Desencadenant (Trigger):** `kube_pod_status_phase{phase="Failed"}` o fallada consecutiva de 3 sondejos HTTP d'estat (*Health Check*).
- **Acció Correctiva:** L'orquestrador (ex. Kubernetes) destrueix el contenidor afectat i en crea un de nou immediatament de manera nativa, utilitzant la imatge base i reassignant l'adreça de xarxa IP.

B. Purga i alliberament automàtic d'espai de disc (*Disk Saturation*)

- **Esdeveniment:** El volum d'emmagatzematge on s'allotja l'aplicació o els *logs* supera un llindar crític de seguretat.
- **Desencadenant (Trigger):** `node_filesystem_free_bytes / node_filesystem_size_bytes < 0.15` (Espai lliure inferior al 15% durant més de 2 minuts).
- **Acció Correctiva:** Es llança un *webhook* cap a un servei d'automatització (ex. *AWX/Ansible* o un script *Cronjob* d'emergència) que executa ordres com `docker system prune -af --volumes` per netejar imatges antigues o esborrar fitxers temporals de fa més de 7 dies.

C. Bloquejos de fils o fuites de memòria en calent (*Memory Leaks / Deadlocks*)

- **Esdeveniment:** L'aplicació comença a consumir memòria de manera progressiva i no l'allibera, provocant una lentitud extrema abans de caure per *OOMKilled*.
- **Desencadenant (Trigger):** L'ús de la memòria RAM supera el 90% i la taxa de peticions HTTP 504 (*Gateway Timeout*) augmenta.
- **Acció Correctiva:** S'executa un script automatitzat que realitza un **bolcat de memòria (heap dump)** per a la seva posterior anàlisi tècnica i, seguidament, realitza un *Rolling Restart* (reinici esglaonat) dels nodes de l'aplicació per alliberar la memòria sense tallar el servei als usuaris.

7.3. Disseny de Seguretat: Evitar Bucles Infinites (*Flapping*)

L'autoreparació automatitzada comporta un risc: si la causa arrel del problema és un error de codi persistent (ex. una base de dades caiguda o un error de sintaxi al codi), l'automatització reiniciarà el contenidor indefinidament, consumint recursos de manera inútil.

Per evitar-ho, cal configurar dues regles de protecció:

- **Llindars de reintent (*Backoff Limit*):** Si l'acció d'autoreparació s'executa **més de 3 vegades en un interval de 15 minuts** sense èxit, el desencadenant s'ha de pausar automàticament.
- **Escalat Humà d'Emergència:** Un cop superat el llindar de reintents, el sistema ha de congelar l'automatització i **enviar una alerta d'alta prioritat** (ex. mitjançant *PagerDuty* o *Opsgenie*) per requerir de manera urgent la intervenció d'un enginyer de guàrdia.

--ooOOoo--